

Ray Tracing in One Weekend

레이 트레이싱 기초부터 렌더링 완성까지

신승우 · 컴퓨터 그래픽스 · 2026년 9월

목차

PART 1 렌더링 기초

- 01 CUDA Toolkit Install
- 02 Output an Image
- 03 The vec3 class
- 04 Rays, a simple camera, and background
- 05 Adding a sphere
- 06 Surface normals and multiple objects
- 07 Moving Camera code into its own class
- 08 Antialiasing

PART 2 재질 표현

- 09 Diffuse Materials
- 10 Metal
- 11 Dielectrics

PART 3 카메라 제어와 최종 결과

- 12 Positionable camera
- 13 Defocus Blur
- 14 Where Next?



RAY TRACING IN ONE WEEKEND

PART 1

렌더링 기초

이미지 출력부터 안티에일리어싱까지

Chapter 01 – 08



안티에일리어싱 적용 결과

CUDA 환경 구성: 드라이버 호환성 확인 후 Toolkit 설치

STEP 1

그래픽 드라이버 업데이트

CUDA의 최소 드라이버 버전 확인
요구 버전보다 낮으면 업데이트



STEP 2

CUDA Toolkit 설치

OS별 설치 파일 다운로드
developer.nvidia.com/cuda-downloads



STEP 3

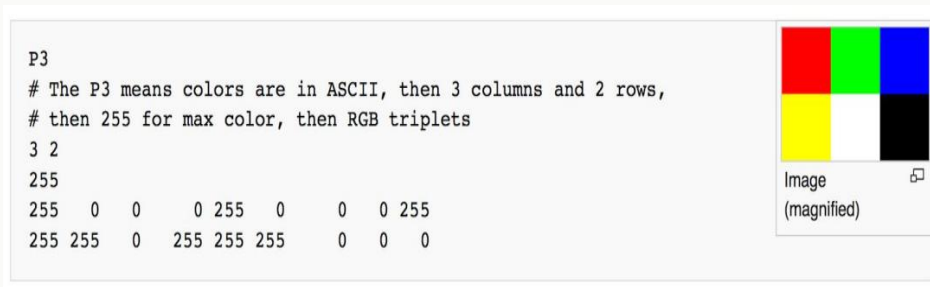
새 프로젝트 생성

설치 시 VS에 CUDA 자동 연동
새 프로젝트에서 CUDA 템플릿 선택

PPM(Portable PixMap) 형식: 헤더 뒤에 RGB 값을 3칸씩 나열

PPM이란?

- 이미지 생성 시, 가장 간단한 형식 중 하나
- 색상은 RGB 3칸씩 끊어서 읽기
- 헤더는 줄바꿈 · 공백 상관없이 구분만 필요



〈그림 1〉 PPM 형식

PPM 헤더 출력 코드

- 헤더로 형식 · 크기 · 최대 색상값 전달

```
std::cout << "P3\n" << ImageWidth << ' ' << ImageHeight << "\n255\n";
```

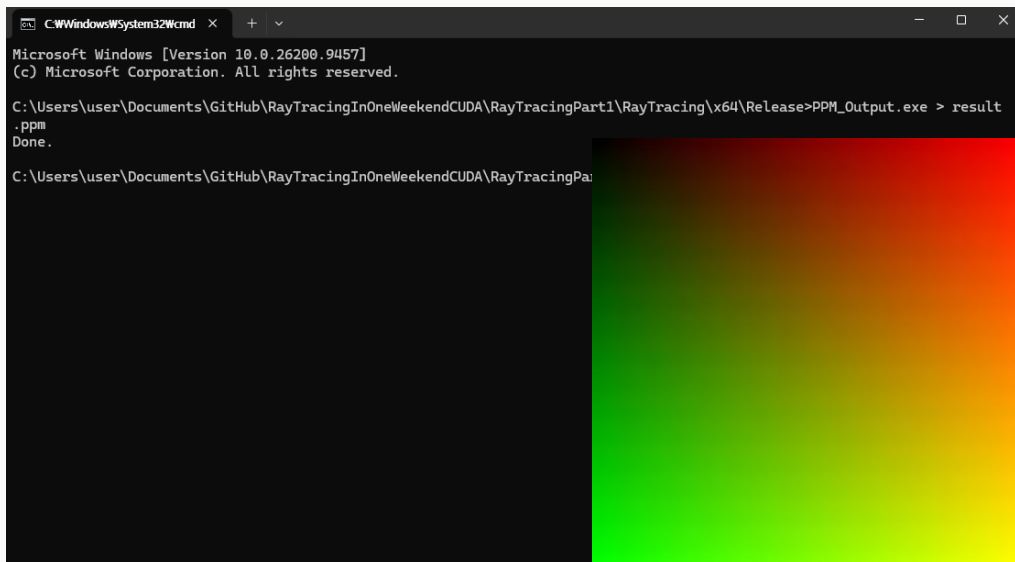
출력 결과

```
P3
512 256
255
... 실제 색상 내용 ex)255 255 255
```

실행과 확인: 파일 저장과 진행 상황 표시

실행 파일을 활용하여 이미지 생성 - 리다이렉션 명령

```
PPM_Output > image.ppm
```



〈그림 2〉 이미지 생성 진행 상황 출력 및 결과

리다이렉션 '>'

- 기호 뒤 파일 이름으로 명령 결과 입력
- 존재하는 파일이면 덮어쓰기, 없는 파일이면 새로 생성

진행 상황 확인 - `std::clog`

- 쉘에서 ppm 생성 시 남은 Line 수 표시
- `std::flush`와 `\r`로 줄바꿈 없이 한 줄 표시
- 실행이 오래 걸리는 최종 결과 단계에서 유효

표현 색상 범위: 헤더에 적은 최대값이 기준

PPM 헤더 출력 코드 변경

```
std::cout << "P3\n" << ImageWidth << ' ' << ImageHeight << "\n900\n";
```

색상 표현 범위

- 최대 16비트까지 표현 가능 [0, 65535]
- 헤더에 최대값을 적기만 하면 적용



〈그림 3〉 최대값 255 출력

최대값 900 예시

- 헤더만 900으로 변경 -> 255/900으로 해석
- 표현 색상이 낮은 값으로, 어두운 부분 출력
- 이후 범위를 0 ~ 255로 재설정 후, 과정 진행



〈그림 4〉 최대값 900 출력

색상 계산: 0~1 정규화 후 255.999 곱셈

정규화 코드

```
auto r = double(i) / (ImageWidth - 1);
auto g = double(j) / (ImageHeight - 1);
auto b = 0.0;
```

색상값 변환 코드

```
int ir = int(255.999 * r);
int ig = int(255.999 * g);
int ib = int(255.999 * b);
```

곱하는 값	r = 1.0 결과	특징
× 255	1.0에서만 255	폭 불균등
× 256	256	유효하지 않은 값
× 255.999	255	256보다 작지만 근접

정규화

- 이미지 크기와 무관하게 균일하게 색칠하기 위함
- 위치에 비례해 [0.0 - 1.0] 비율로 값 변환

색상 값 변환 (255.999 사용)

- 헤더의 최대 색상값(255) 범위에 맞추기 위함
- int()는 버림 방식이며, 0 ~ 255를 256등분 필요

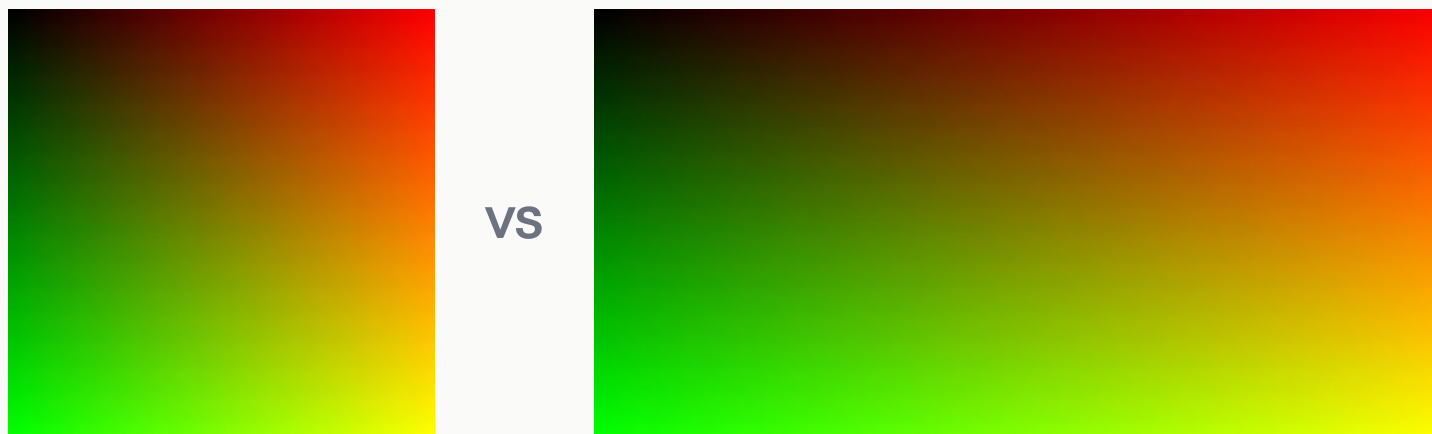
이미지 해상도 변화에도 색상 비율 유지

이미지 해상도 변경 후 비교

- 해상도 변화와 색상 비율의 관계를 확인하기 위함
- 크기 256×256 과 512×256 비교

결과

- 픽셀 수가 늘어나도 **정규화**로 인해 같은 비율 유지
- 같은 비율 위치에 같은 색 배정



〈그림 5〉 256×256과 512×256 이미지 비교

Vector3: 헤더 파일로 만든 사용자 정의 자료형

Vector3 정의 – vec3.h

```
#ifndef VEC3_H
#define VEC3_H

struct Vector3
{
    double E[3];
    Vector3() : E{ 0, 0, 0 } {}
    Vector3(double e0, double e1, double e2)
        : E{ e0, e1, e2 } {}
    double X() const { return E[0]; }
    ...
};
typedef Vector3 Vec3;
using Point3 = Vector3;

#endif
```

헤더 파일 제작

- 여러 .cpp 파일에서 포함하기 위해 헤더로 제작

중복 포함 방지

방식	기준
#ifndef · #define · #endif	같은 매크로(VEC3_H) 정의 여부
#pragma once	파일 자체

멤버 초기화 리스트

- 생성과 동시에 초기화하기 위해 사용
- 원문 코드는 생성 후 대입 방식 사용

멤버 함수와 일반 함수: 일반 함수는 inline으로 정의

멤버 함수

- 인스턴스 자신이 함수 파라미터인 경우

```
struct Vector3 {  
    Vector3 operator-() const  
    {  
        return Vector3(-E[0], -E[1], -E[2]);  
    }  
};
```

inline 사용 이유

- 일반 함수를 여러 .cpp 파일에서 포함하면 중복 정의 오류 발생
- **inline** 함수는 여러 .cpp 파일에 같은 함수 정의 존재 허용

일반 함수 (inline)

- 임의의 Vector3 인스턴스 간 연산인 경우

```
inline Vector3 operator+(const Vector3& u,  
                        const Vector3& v)  
{  
    return Vector3(u.E[0] + v.E[0], u.E[1] + v.E[1],  
                  u.E[2] + v.E[2]);  
}
```

inline 미사용 시 링킹 단계에서 중복 정의 오류

test.h

```
#pragma once
int Add(int a, int b)
{
    return a + b;
}
```

Main.cpp

```
#include "test.h"
int Test();
int main()
{
    std::cout << Add(1, 2) << '\n';
    std::cout << Test() << '\n';
}
```

원인

- 각 .cpp 파일은 개별적으로 컴파일
- 링킹 단계에서 동일 함수 중복 정의 문제 발생

```
LNK2005"int __cdecl Add(int,int)" (?Add@@YAHHH@Z) already defined in Main.obj
LNK1169one or more multiply defined symbols found
```

〈그림 6〉 링킹 단계 중복 정의 오류

Test.cpp

```
#include "test.h"
int Test()
{
    return Add(3, 4);
}
```

주요 벡터 유틸리티 함수: Dot · UnitVector · Cross

Dot Product (= Scalar Product)

$a \cdot b = a_0b_0 + a_1b_1 + a_2b_2 \rightarrow$ 스칼라

```
inline double Dot(const Vector3& u,
                  const Vector3& v)
{
    return u.E[0] * v.E[0] + u.E[1] * v.E[1]
           + u.E[2] * v.E[2];
}
```

UnitVector 활용

$v / |v|, |v| = \sqrt{v_0^2 + v_1^2 + v_2^2} \rightarrow$ 벡터

```
inline Vector3 UnitVector(const Vector3& v)
{
    return v / v.Length();
}
```

Cross Product

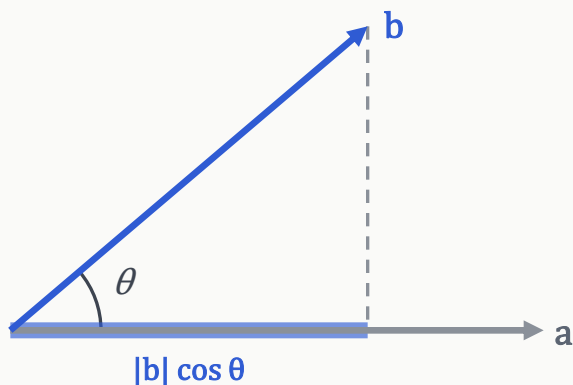
$a \times b = (a_1b_2 - a_2b_1, a_2b_0 - a_0b_2, a_0b_1 - a_1b_0) \rightarrow$ 벡터

```
inline Vector3 Cross(const Vector3& u, const Vector3& v)
{
    return Vector3(u.E[1] * v.E[2] - u.E[2] * v.E[1],
                  u.E[2] * v.E[0] - u.E[0] * v.E[2],
                  u.E[0] * v.E[1] - u.E[1] * v.E[0]);
}
```

주요 벡터 연산의 의미: Dot · UnitVector · Cross

Dot Product

- 크기와 사이 각 정보, 부호로 방향 판별
- 자기 자신과 내적 시 길이의 제곱

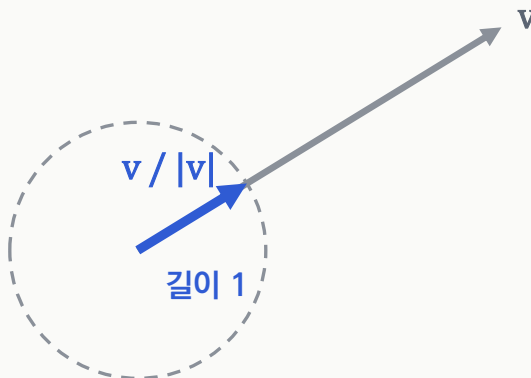


$$a \cdot b = |a| \times |b| \cos \theta$$

$$a \cdot a = |a|^2 \rightarrow |a| = \sqrt{a \cdot a}$$

UnitVector

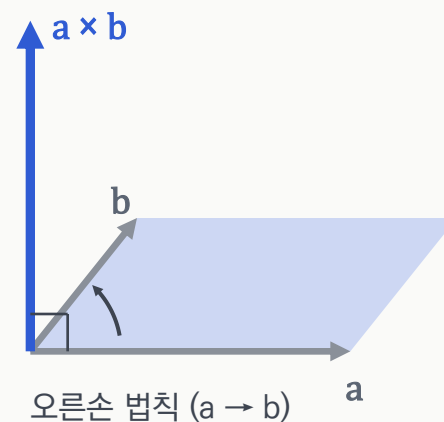
- 길이가 1인 벡터
- 크기 정보 제거, 방향 정보만 유지



$$\text{UnitVector}(v) = v / |v|, \text{ 길이 } 1$$

Cross Product

- 두 벡터에 모두 수직인 벡터
- 크기는 a, b가 만드는 평행사변형 넓이



$$|a \times b| = \text{평행사변형 넓이}$$

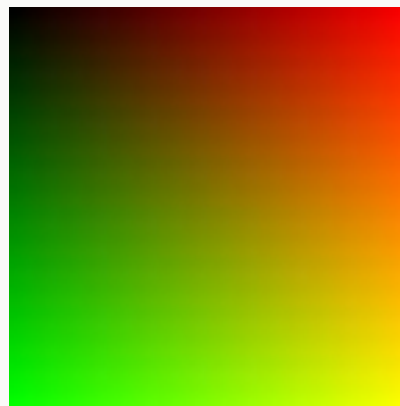
Vec3를 Color에 적용: 결과는 앞 장과 동일

color.h

```
#include "vec3.h"
using Color = Vec3;
void WriteColor(std::ostream& out,
               const Color& pixelColor)
{
    auto r = pixelColor.X();
    auto g = pixelColor.Y();
    auto b = pixelColor.Z();
    int rByte = int(255.999 * r);
    int gByte = int(255.999 * g);
    int bByte = int(255.999 * b);
    out << rByte << ' ' << gByte
        << ' ' << bByte << '\n';
}
```

main 픽셀 출력

```
auto PixelColor = Color(
    double(i) / (ImageWidth - 1),
    double(j) / (ImageHeight - 1), 0);
WriteColor(std::cout, PixelColor);
```



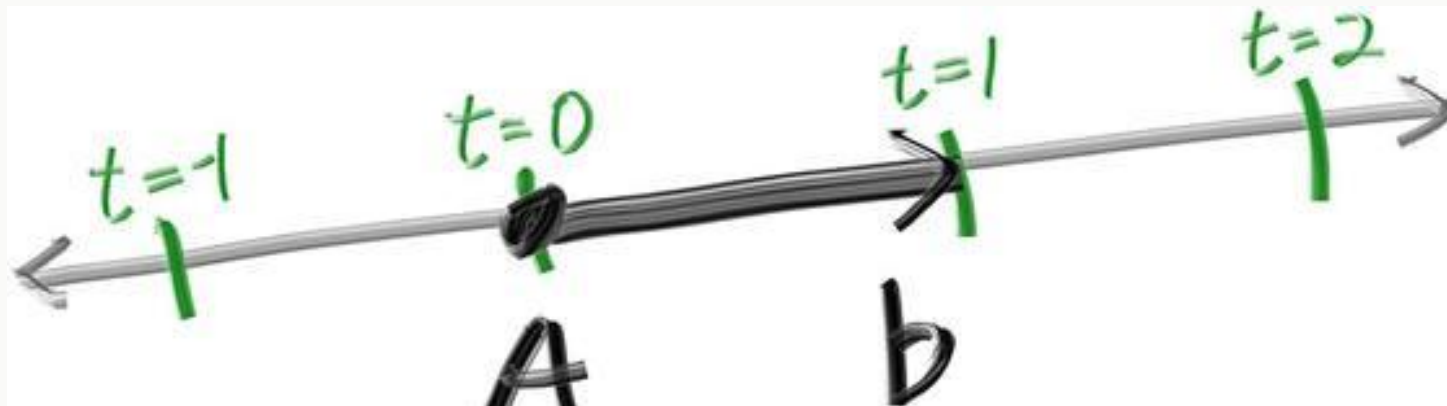
〈그림 7〉 Vec3 적용 결과

광선: $P(t) = A + tb$ 로 광선 위 지점 좌표 계산

광선 식

$$P(t) = A + t b$$

- A (Point): 광선의 원점
- b (Direction): 광선의 방향
- t: 광선 이동량



〈그림 8〉 광선 $P(t) = A + tb$

Ray 클래스: 원점과 방향을 const &로 반환

Ray.h

```
class Ray
{
public:
    Ray() {}
    Ray(const Point3& origin, const Vector3& direction)
        : mOrigin(origin), mDirection(direction) {}
    const Point3& Origin() const { return mOrigin; }
    const Vector3& Direction() const { return mDirection; }
    Point3 At(double t) const
    {
        return mOrigin + t * mDirection;
    }
private:
    Point3 mOrigin;
    Vector3 mDirection;
};
```

참조(&)

- 변수에 별명을 부여하는 것과 같음
- 선언과 동시에 참조 필요
- 참조 대상 변경 불가

const & 반환 방식

- const &로 받으면 수정 · 대상 변경 불가
- 동일 자료형으로 복사하여 받기 가능

이미지 · 뷰포트 크기: 실제 이미지 비율로 뷰포트 설정

이미지 크기 설정

- 기준 크기 하나를 정한 뒤, 종횡비로 나머지 설정
- **이미지 크기** = 뷰포트를 몇 칸으로 나눌지를 의미

뷰포트 크기 설정

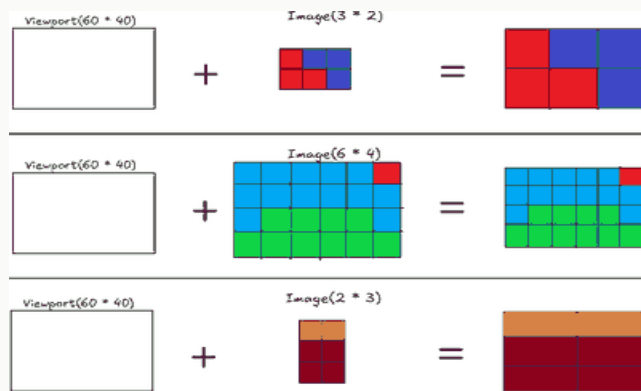
- 이상적 비율 대신 실제 이미지 비율 사용
- 비율 불일치 시 이미지 왜곡 발생

```
double aspectRatio = 16.0 / 9.0;
int imageWidth = 400;
int imageHeight = int(imageWidth / aspectRatio);
imageHeight = (imageHeight < 1) ? 1 : imageHeight;
```

```
double viewportHeight = 2.0;
double viewportWidth = viewportHeight
    * (double(imageWidth) / imageHeight);
```



〈그림 9〉 이미지 비율과 최소 높이 보장



〈그림 10〉 이미지 크기에 따른 뷰포트 분할

픽셀 중앙으로 Ray 발사: 좌상단 기준 시 편향 발생

픽셀 기준점

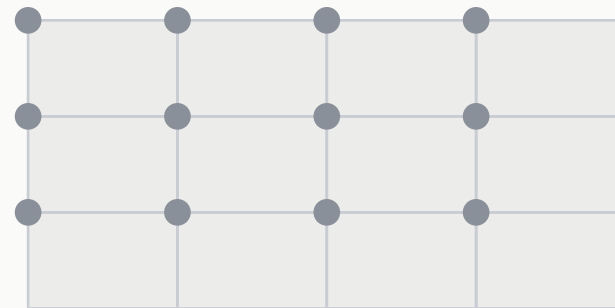
- 뷰포트 좌표는 범위 내 아무 곳이나 사용 가능
- 이미지에서는 픽셀이 최소 단위

Ray 방향

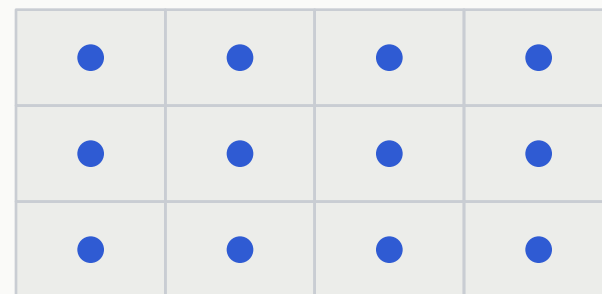
- 카메라 중심점에서 각 픽셀 중심점으로 Ray발사

중앙을 쓰는 이유

- 좌상단 기준 시 모든 픽셀이 편향 출력되기 때문
- 중앙에서 구한 색이 전체 평균색을 보장하진 않음



좌상단 기준: 한쪽으로 편향



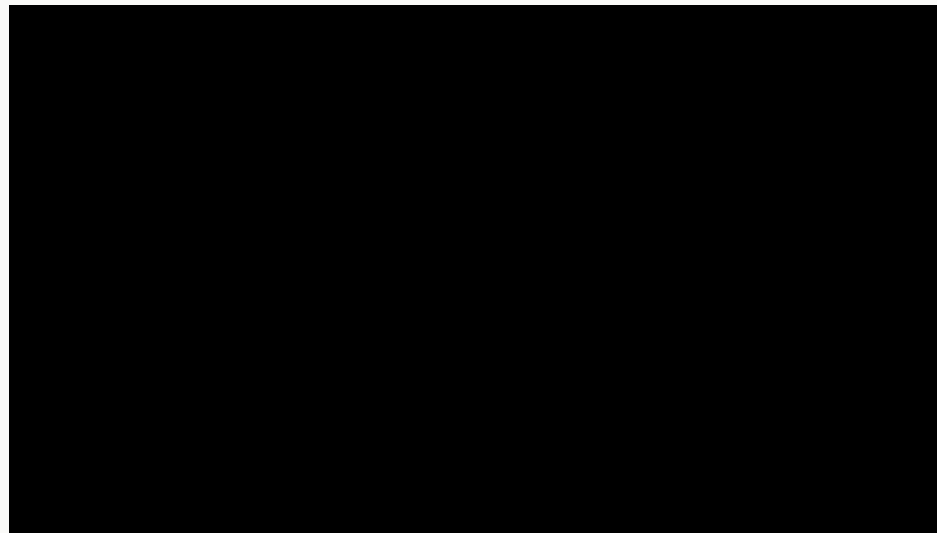
픽셀 중앙 기준

결과 1: 모든 광선에 검은색 반환

RayColor · 검정 이미지

- 모든 광선에 대해 검은색 반환

```
Color RayColor(const Ray& r)
{
    return Color(0, 0, 0);
}
```



〈그림 11〉 검정 이미지 출력

결과 2: 정규화한 광선 방향으로 색 선형 보간

선형 보간 (Lerp)

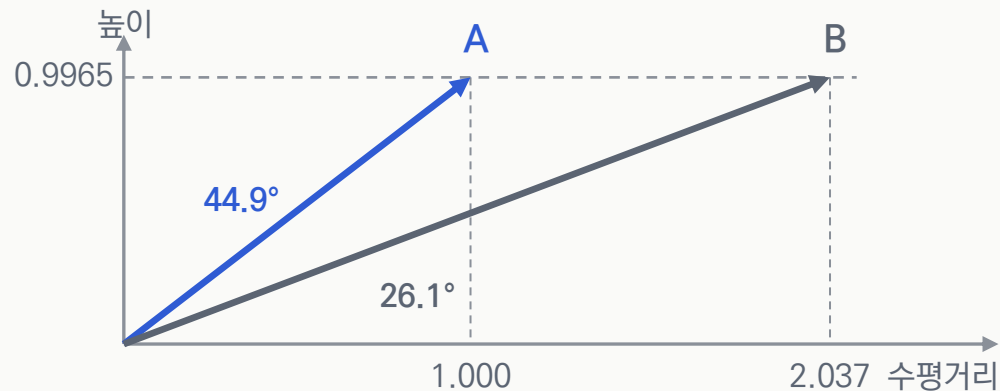
- 광선 방향에 따라 픽셀의 최종 색 결정

$$\text{blendedValue} = (1 - a) \cdot \text{startValue} + a \cdot \text{endValue}$$

정규화 (Normalized)

- 벡터의 방향 성분만 남기기 위해 사용
- 같은 y값이어도 x, z에 따라 방향이 다름
- 벡터 크기를 1로 통일

$$\mathbf{d} = |\mathbf{d}| \cdot \hat{\mathbf{d}} \quad (\text{크기} \cdot \text{방향})$$



〈그림 12〉 같은 높이, 다른 방향

결과 2: y 값을 $[0, 1]$ 로 보정한 흰색-파랑 그래디언트 이미지

그래디언트 이미지

- $[-1, 1]$ 정규화 결과를 색상 범위 $[0, 1]$ 로 변환
- 1을 더해 $[0, 2]$ 로 만든 뒤 0.5 곱하기
- y 가 높을수록 파란색, 낮을수록 흰색

```
Color RayColor(const Ray& r)
{
    Vec3 unitDirection = UnitVector(r.Direction());
    auto a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0);
}
```



〈그림 13〉 그래디언트 이미지 출력

구 방정식: $(P - C) \cdot (P - C) = r^2$ 이면 구의 표면 위

구의 표면

- 중심에서의 거리가 반지름과 같은 경계
- 구의 표면을 기준으로 색상을 칠해 표현

원점 중심, 반지름 r 인 구

$$x^2 + y^2 + z^2 = r^2 \rightarrow \text{표면 위}$$

$$x^2 + y^2 + z^2 < r^2 \rightarrow \text{내부}$$

$$x^2 + y^2 + z^2 > r^2 \rightarrow \text{외부}$$

중심을 C 로 옮긴 반지름 r 인 구

$$(C_x - x)^2 + (C_y - y)^2 + (C_z - z)^2 = r^2$$

$$\Rightarrow (P - C) \cdot (P - C) = r^2$$

기호 정리

기호	뜻	코드
Q	광선의 출발점 (카메라 위치)	<code>r.Origin()</code>
d	광선의 방향 벡터 (단위 벡터 불필요)	<code>r.Direction()</code>
t	광선 위 이동량을 나타내는 실수	-
$P(t)$	광선 위의 점 $P(t) = Q + td$	<code>r.At(t)</code>
C	구의 중심	<code>center</code>
r	구의 반지름	<code>radius</code>

광선-구 교점: 광선 식 대입으로 t에 대한 이차방정식 도출

1. 구 표면 조건

점 P에서 중심 C로 가는 벡터 $C - P$

$$(C - P) \cdot (C - P) = r^2$$

*제곱이라 부호 무관, $C - Q$ 를 얻기 위해 선택

2. 광선 위의 점 대입

$P(t) = Q + td$ 활용

$$(C - (Q + td)) \cdot (C - (Q + td)) = r^2$$

3. t항과 나머지 분리

$C - Q$: 출발점 $\rightarrow$ 구 중심 벡터

$$(-td + (C - Q)) \cdot (-td + (C - Q)) = r^2$$

4. 분배 법칙으로 내적 전개

$(a + b) \cdot (a + b) = a \cdot a + 2a \cdot b + b \cdot b$

$$t^2(d \cdot d) - 2td \cdot (C - Q) + (C - Q) \cdot (C - Q) = r^2$$

- 스칼라는 곱해서 앞으로, 벡터는 내적 형태 그대로 유지

이차방정식 계수: a는 방향 길이 제곱, b는 방향 일치도, c는 구 안팎 판별

$$t^2(d \cdot d) - 2t d \cdot (C - Q) + (C - Q) \cdot (C - Q) - r^2 = 0 \Rightarrow at^2 + bt + c = 0$$

a: 방향 벡터 길이의 제곱

$$a = d \cdot d = |d|^2$$

- 항상 양수
- 단위 벡터면 1

b: 광선 방향과 중심 방향

$$b = -2 d \cdot (C - Q)$$

- 두 방향이 얼마나 같은 쪽인지 표현

예) 카메라 원점, $d = (0, 0, -1)$

구 위치	$d \cdot (C - Q)$	b
앞 (0, 0, -1)	1	-2
옆 (1, 0, 0)	0	0
뒤 (0, 0, 1)	-1	2

c: 출발점과 구의 위치 관계

$$c = (C - Q) \cdot (C - Q) - r^2 \\ = |C - Q|^2 - r^2$$

- 출발점이 구 밖이면 양수
- 출발점이 구 안이면 음수

판별식의 부호로 광선과 구의 교점 개수 판별

근의 공식

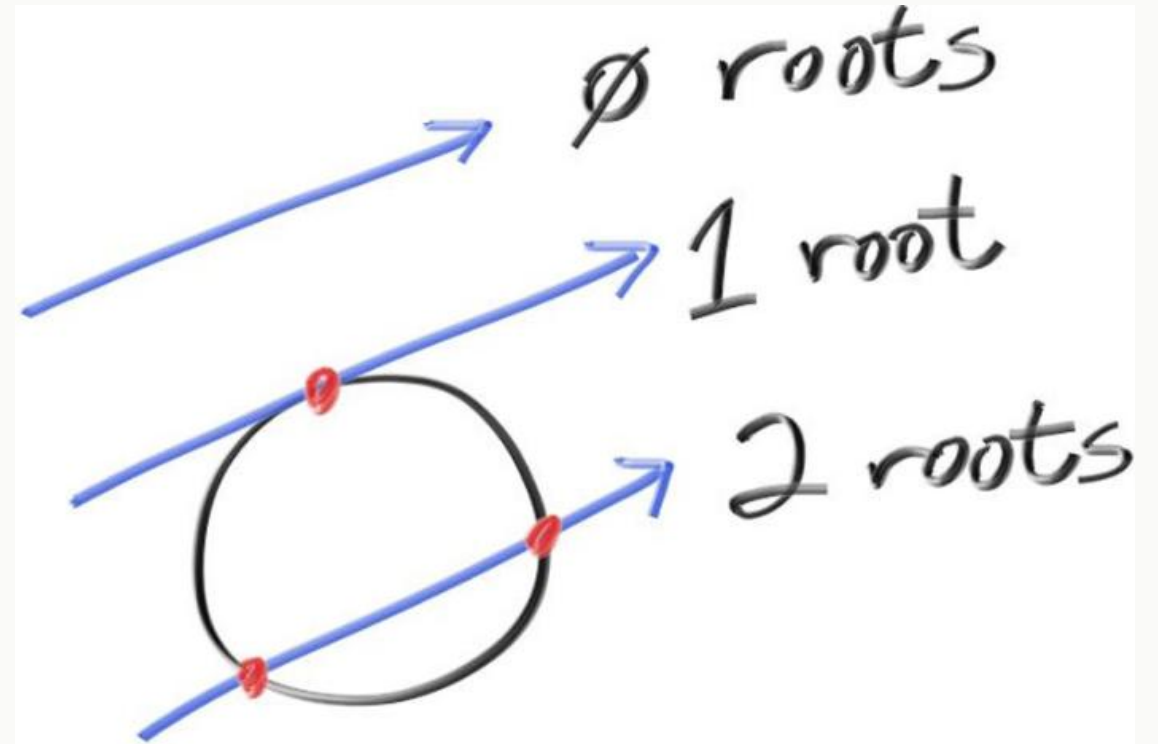
- t에 대한 이차방정식이므로 근의 공식 적용 가능

$$t = (-b \pm \sqrt{D}) / 2a$$

$$D = b^2 - 4ac$$

D의 부호와 교점

D	근	기하학적 의미
$D > 0$	서로 다른 실근 2개	광선이 구를 뚫고 지나감
$D = 0$	중근 1개	광선이 구에 스치듯 접함
$D < 0$	실근 없음	광선이 구를 빗나감



〈그림 14〉 광선과 구의 교점 경우

HitSphere: 판별식 $D \geq 0$ 이면 광선이 구와 충돌

HitSphere · 교점 판별

```
bool HitSphere(const Point3& center,
               double radius, const Ray& r)
{
    Vec3 cToO = center - r.Origin();
    double a = Dot(r.Direction(), r.Direction());
    double b = -2.0 * Dot(r.Direction(), cToO);
    double c = Dot(cToO, cToO) - radius * radius;
    double discriminant = b * b - 4 * a * c;
    return (discriminant >= 0);
}
```

RayColor · 구에 맞으면 빨간색

```
Color RayColor(const Ray& r)
{
    if (HitSphere(Point3(0, 0, -1), 0.5, r))
    {
        return Color(1, 0, 0);
    }
    Vec3 unitDirection = UnitVector(r.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0);
}
```

결과: 교점 개수와 무관하게 구 전체를 같은 빨간색으로 출력

빨간 구 출력

- $(0, 0, -1)$ 에 반지름 0.5인 구 배치
- 구와 만나면 빨간색, 아니면 배경 그라디언트

한계

- 교점이 2개여도 별다른 처리 없음
- 구의 색상이 항상 동일하게 표시



〈그림 15〉 구 추가 결과

법선 벡터: 중심에서 충돌점을 향한 벡터를 반지름으로 나눈 단위 벡터

단위 법선

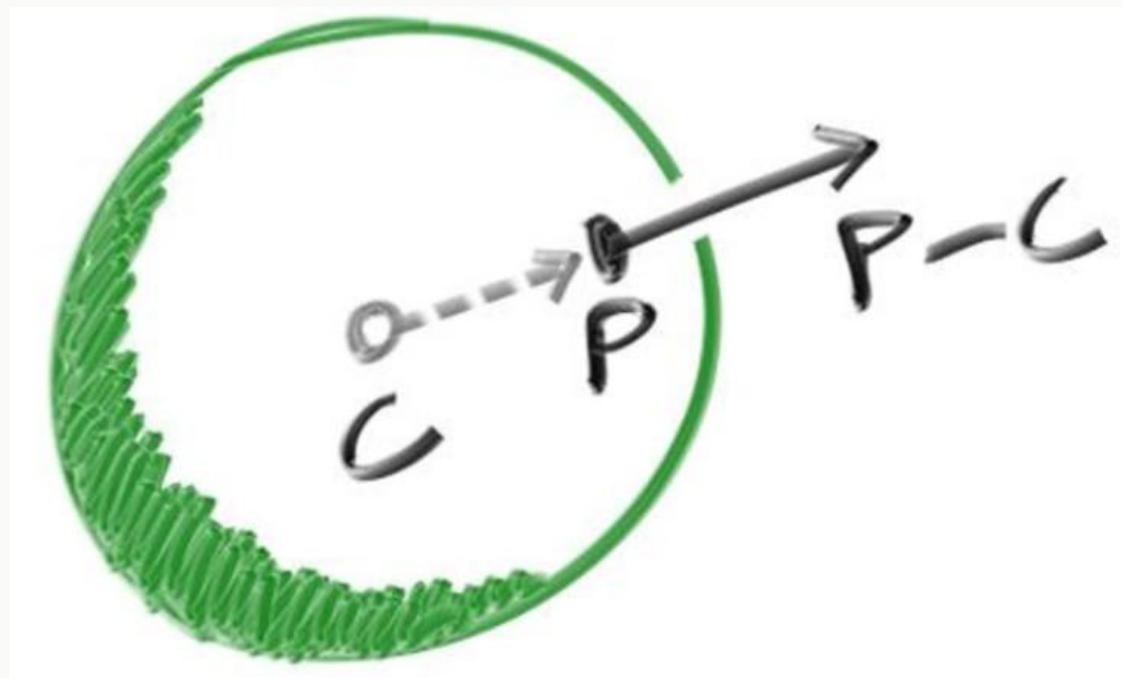
- C에서 P로 가는 화살표의 길이는 r
- 정규화: 벡터를 자기 길이로 나눠 길이를 1로 만들

$$\hat{v} = v / |v|$$
$$\hat{n} = (P - C) / r$$

법선을 색으로 표현하기

- 색은 0~1 범위, 단위 법선 성분은 $[-1, 1]$
- $0.5(n + 1)$ 적용으로 범위를 $[0, 1]$ 로 변환

$$\text{color} = 0.5(n + 1)$$



〈그림 16〉 구의 단위 법선 벡터

HitSphere: 두 근 중 작은 t로 가장 가까운 충돌점 반환

가장 가까운 충돌점

- 가장 작은 t로 최종 색상 결정
- $t > 0.0$: 카메라 뒤쪽 충돌 제외
- 근의 공식 중 작은 근 사용
- $a > 0$ 이므로 $-\sqrt{D}$ 쪽 근이 항상 작음

$$t = (-b - \sqrt{D}) / 2a$$

t의 의미

- 작은 t: 카메라에 가까운 구의 **앞면**
- 큰 t: 구를 뚫고 나가는 **뒷면**

HitSphere · 충돌점 t 반환

```
double HitSphere(const Point3& center, double radius, const Ray& r)
{
    Vec3 originToCenter = center - r.Origin();
    double a = Dot(r.Direction(), r.Direction());
    double b = -2.0 * Dot(r.Direction(), originToCenter);
    double c = Dot(originToCenter, originToCenter) - radius * radius;
    double discriminant = b * b - 4 * a * c;
    // 실제 충돌점을 구하기 위해 근의 공식 전체 사용
    if (discriminant < 0.0)
    {
        return -1.0;
    }
    return (-b - std::sqrt(discriminant)) / (2.0 * a); // 가까운 충돌 지점
}
```

RayColor: 충돌점의 법선 성분을 색상으로 표현

RayColor · 법선 시각화

- 법선 벡터 시각화를 위해 각 축 성분 기반으로 색상 부여

```
Color RayColor(const Ray& ray)
{
    double t = HitSphere(Point3(0.0, 0.0, -1.0), 0.5, ray);
    if (t > 0.0)
    {
        Vec3 surfaceNormal = UnitVector(ray.At(t)
                                         - Vec3(0.0, 0.0, -1.0));
        return (0.5 * Color(surfaceNormal.X() + 1.0,
                           surfaceNormal.Y() + 1.0,
                           surfaceNormal.Z() + 1.0));
    }
    Vec3 unitDirection = UnitVector(ray.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
}
```

결과



〈그림 17〉 법선 벡터 시각화 결과

코드 정리 1- 'b = -2h' 치환으로 근의 공식 축소

h 치환

$$h = b / -2 = d \cdot (C - Q)$$

- 짝수 근의 공식에서 $b' = -h$ 로 둔 것과 동일

판별식 변화

$$\begin{aligned} t &= (-b \pm \sqrt{b^2 - 4ac}) / 2a \\ &= (2h \pm \sqrt{4h^2 - 4ac}) / 2a \\ &= (2h \pm 2\sqrt{h^2 - ac}) / 2a \\ &= (h \pm \sqrt{h^2 - ac}) / a \end{aligned}$$

- $\sqrt{\quad}$ 안의 $h^2 - ac$ 가 새 판별식

HitSphere · 단순화 적용

```
double HitSphere(const Point3& center,
                 double radius, const Ray& ray)
{
    Vec3 originToCenter = center - ray.Origin();
    double a = ray.Direction().LengthSquared();
    double h = Dot(ray.Direction(), originToCenter);
    double c = originToCenter.LengthSquared()
               - radius * radius;
    double discriminant = h * h - a * c;
    if (discriminant < 0)
    {
        return -1.0;
    }
    return (h - std::sqrt(discriminant)) / a;
}
```

코드 정리 2- Hittable로 충돌 가능한 객체 통합

충돌 가능한 객체 추상화

- 충돌 가능한 모든 것을 추상 클래스로 정의
- 여러 구를 모두 Hittable 객체로 처리
- $t_{\min} < t < t_{\max}$ 일 때만 충돌로 계산

HitRecord 저장 방식

- 충돌마다 법선 등을 바로 계산해 저장
- 더 가까운 충돌 시 앞의 계산은 버려짐
- 단순함을 위해 이 방식 선택

HitRecord · Hittable

```
class HitRecord
{
public:
    Point3 P;
    Vec3 Normal;
    double T;
};

class Hittable
{
public:
    virtual bool Hit(const Ray& ray,
                    const Interval& rayT,
                    HitRecord& hitRecord) const = 0;
};
```

World: HittableList에 구 2개를 넣어 함께 렌더링

진행 흐름

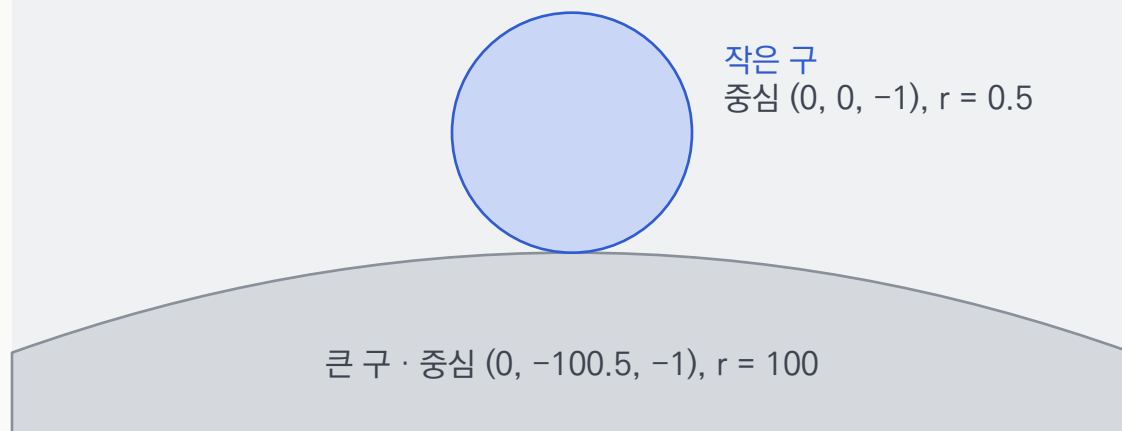
- HittableList 인스턴스로 **World** 생성
- 충돌 가능한 물체를 World에 추가
- 구 2개 추가 후 렌더링

World 구성

```
// World
HittableList world;
world.Add(std::make_shared<Sphere>(
    Point3(0.0, 0.0, -1.0), 0.5));
world.Add(std::make_shared<Sphere>(
    Point3(0.0, -100.5, -1.0), 100.0));
```

예상 결과

카메라에서 본 화면



〈그림 18〉 예상 결과: 화면 중앙의 작은 구와 바닥의 큰 구

RayColor: 구 1개 대신 World 전체와 충돌 검사

픽셀 루프

```
for (int pixelIndex = 0; pixelIndex < mImageWidth;
    ++pixelIndex)
{
    Point3 currentPixelCenter = mFirstPixelCenter
        + (pixelIndex * mPixelDeltaU)
        + (scanlineIndex * mPixelDeltaV);
    Vec3 rayDirection = currentPixelCenter - mCenter;
    Ray ray(mCenter, rayDirection);
    Color pixelColor = RayColor(ray, world);
    WriteColor(std::cout, pixelColor);
}
```

RayColor · World 충돌 검사

```
Color RayColor(const Ray& ray, const Hittable& world)
{
    HitRecord hitRecord;
    if (world.Hit(ray, Interval(0.0, Infinity),
        hitRecord))
    {
        return (0.5 * (hitRecord.Normal
            + Color(1.0, 1.0, 1.0)));
    }
    Vec3 unitDirection = UnitVector(ray.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0);
}
```

- RayColor: 빈 HitRecord 생성 후 **world.Hit()**으로 0~무한대 사이 충돌 물체 확인
- 충돌 시 기록된 법선으로 색 계산, 충돌이 없으면 배경색 반환

HittableList::Hit – 가장 가까운 물체만 충돌 기록

목록 순회

- world.Hit() = HittableList의 Hit()
- 목록의 구를 차례로 돌며 각 구의 Hit() 호출

가장 가까운 충돌 유지

- 충돌 시 그 t를 closestSoFar로 저장
- 다음 물체는 closestSoFar를 최대 범위로 검사
- 더 먼 물체는 자동 제외
- 목록 순회 후 가장 가까운 물체 기록만 남음
- object→Hit()으로 Sphere::Hit 호출

HittableList::Hit

```
bool Hit(const Ray& ray, double rayTMin, double rayTMax,
        HitRecord& hitRecord) const override
{
    HitRecord temporaryHitRecord;
    bool bHitAnything = false;
    auto closestSoFar = rayTMax;
    for (const auto& object : mObjects)
    {
        if (object->Hit(ray, rayTMin, closestSoFar,
                        temporaryHitRecord))
        {
            bHitAnything = true;
            closestSoFar = temporaryHitRecord.T;
            hitRecord = temporaryHitRecord;
        }
    }
    return bHitAnything;
}
```

Sphere::Hit - 유효 범위 안의 근만 충돌로 인정

근 선택 규칙

$$\text{rayTMin} < t < \text{rayTMax}$$

경우	처리
가까운 근 범위 안	그대로 사용
가까운 근 범위 밖	먼 근으로 교체 후 재검사
먼 근도 범위 밖	false 반환

범위 밖 원인

원인	처리
$t \leq \text{rayTMin}$	카메라가 구 안 → 먼 근 사용 구 전체가 카메라 뒤 → false
$t \geq \text{rayTMax}$	더 가까운 물체 존재 → false

Sphere::Hit · 근 선택과 기록

```
// 가까운 근 계산
double squareRootDiscriminant = std::sqrt(discriminant);
double root = (h - squareRootDiscriminant) / a;

// 범위 검사 후 근 선택
if (root <= rayTMin || rayTMax <= root)
{
    root = (h + squareRootDiscriminant) / a;
    if (root <= rayTMin || rayTMax <= root)
    {
        return false;
    }
}

// 충돌 기록 작성
hitRecord.t = root;
hitRecord.point = ray.At(hitRecord.t);
hitRecord.normal = (hitRecord.point - mCenter) / mRadius;
return true;
```

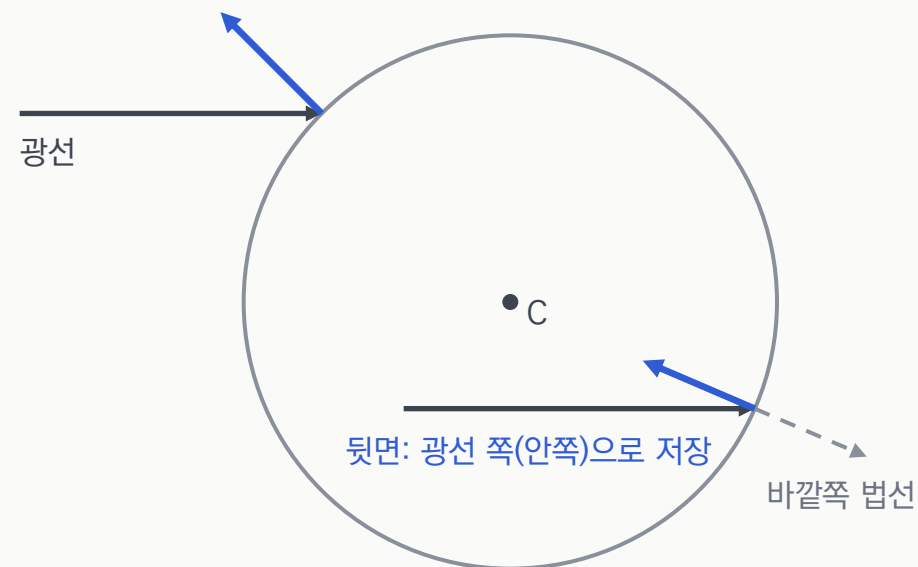
앞면 vs 뒷면: 법선은 항상 광선과 마주 보는 방향으로 저장

Front Faces VS Back Faces

- 법선은 원래 바깥쪽 고정, 충돌 시 광선을 마주 보도록 방향만 맞춰 저장

항목	내용
원래 법선	$(P - C) / r$, 항상 바깥쪽 (구 데이터는 그대로)
판정	광선 방향 · 바깥쪽 법선 $< 0 \rightarrow$ 앞면, $> 0 \rightarrow$ 뒷면
저장	앞면은 그대로, 뒷면은 -법선 (광선이 온 쪽 방향)
이유	반사 · 굴절 공식은 광선이 온 쪽을 향하는 법선 필요
기록	앞면 여부는 bFrontFace에 따로 저장

앞면: 법선 그대로 저장



〈그림 19〉 앞면과 뒷면의 법선 방향

SetFaceNormal: 내적 부호로 앞면을 판정한 뒤 법선 저장

적용 순서

- 1 Sphere::Hit에서 바깥쪽 법선 $(P - C) / r$ 을 구해 SetFaceNormal() 호출
- 2 SetFaceNormal에서 광선 방향과 내적해 부호 확인
- 3 음수면 그대로, 양수면 -outwardNormal로 저장

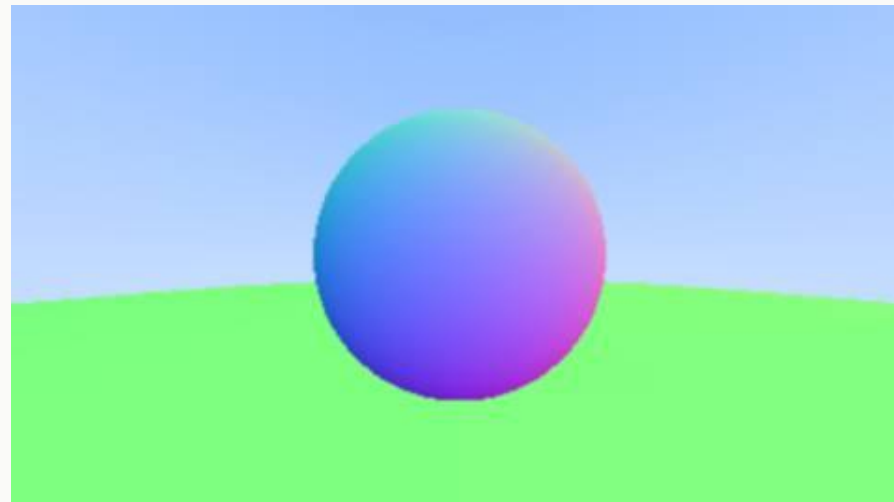
Sphere::Hit

```
hitRecord.T = result;  
hitRecord.P = ray.At(hitRecord.T);  
Vec3 outwardNormal = (hitRecord.P - mCenter) / mRadius;  
hitRecord.SetFaceNormal(ray, outwardNormal);
```

HitRecord::SetFaceNormal

```
void SetFaceNormal(const Ray& ray, const Vec3& outwardNormal)  
{  
    bFrontFace = Dot(ray.Direction(), outwardNormal) < 0;  
    Normal = bFrontFace ? outwardNormal : -outwardNormal;  
}
```

결과



〈그림 20〉 구 2개 렌더링 결과

코드 정리 1- Interval로 t 범위 검사를 한 클래스에 통합

Interval 클래스

- 최솟값과 최댓값을 가진 구간 클래스
- 두 값을 하나로 묶을 뿐 진행 흐름은 동일

멤버	의미
Contains	경계 포함 ($\text{Min} \leq x \leq \text{Max}$)
Surrounds	경계 제외 ($\text{Min} < x < \text{Max}$), 근 선택에 사용
Empty	$(+\infty, -\infty)$
Universe	$(-\infty, +\infty)$

Interval 적용 후 바뀌는 부분

```
// Sphere::Hit 안
double root = (h - squareRootDiscriminant) / a;
if (!rayT.Surrounds(root))
{
    root = (h + squareRootDiscriminant) / a;
    if (!rayT.Surrounds(root))
        return false;
}

// HittableList::Hit 안
Interval currentRayT(rayT.Min, closestSoFar);
if (object->Hit(ray, currentRayT,
                temporaryHitRecord)) { ... }

// RayColor 안
if (world.Hit(ray, Interval(0.0, Infinity),
               hitRecord)) { ... }
```

Camera 클래스로 카메라, 렌더링 분리

Camera 클래스

- main의 카메라·렌더링 코드와 RayColor() 이동
- 기능 변화 없는 리팩토링, 결과는 6장과 동일

구분	멤버
public 변수	aspectRatio, imageWidth
public 함수	Render()
private 함수	Initialize(), RayColor()
private 변수	mImageHeight 등, Initialize()에서 계산

Camera 클래스 구조

```
class Camera
{
public:
    double aspectRatio = 1.0;
    int imageWidth = 100;
    void Render(const Hittable& world);

private:
    void Initialize();
    Color RayColor(const Ray& ray,
                  const Hittable& world) const;

    int mImageHeight = 0;
    Point3 mCenter;
    Point3 mPixel00Location;
    Vec3 mPixelDeltaU;
    Vec3 mPixelDeltaV;
};
```

초기값 설정 후 Render() 호출

진행 흐름

- 기본 생성 → public 변수 대입 → Render() 호출
- Initialize()는 Render() 시작 시 자동 호출
- main에는 world 구성과 카메라 설정만 남음
- RayColor() 이후 충돌 흐름은 6장과 동일

main · Camera::Render

```
// main
HittableList world;
world.Add(std::make_shared<Sphere>(
    Point3(0.0, 0.0, -1.0), 0.5));
world.Add(std::make_shared<Sphere>(
    Point3(0.0, -100.5, -1.0), 100.0));

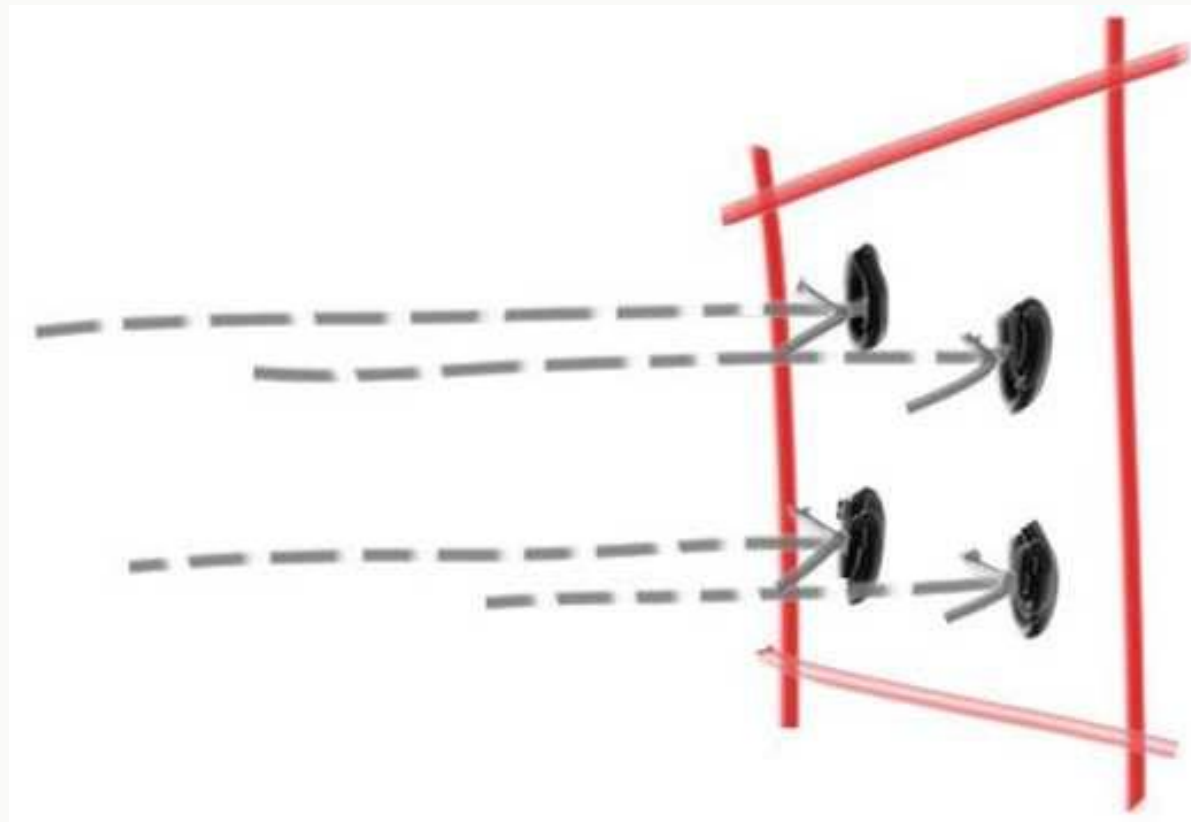
Camera camera;
camera.aspectRatio = 16.0 / 9.0;
camera.imageWidth = 400;
camera.Render(world);

// Camera::Render
void Render(const Hittable& world)
{
    Initialize();
    ...
}
```

샘플링 범위: 픽셀 중앙 기준 $[-0.5, 0.5)$ 오프셋

광선 발사 위치 범위

- 픽셀 한 칸 = 1단위, $[0.0, 1.0)$ 범위 랜덤 샘플링
=> 0.5를 빼서 $[-0.5, 0.5)$ 범위로 이동
=> 픽셀 중앙 0.0 기준 상하좌우 샘플링
- $mPixelDeltaU \cdot V$ 를 곱해 뷰포트 위치로 변환



〈그림 21〉 픽셀 내부 여러 지점으로 광선 발사

랜덤 함수: RandomDouble()로 샘플링용 난수 생성

랜덤 함수 추가

- RandomDouble(): [0, 1) 범위 난수 반환
- RandomDouble(min, max): [min, max) 범위로 변환

rtweekend.h

```
// rtweekend.h
inline double RandomDouble()
{
    // [0, 1)
    // return std::rand() / (RAND_MAX + 1.0);
    static std::uniform_real_distribution<double>
        distribution(0.0, 1.0);
    static std::mt19937 generator;
    return distribution(generator);
}

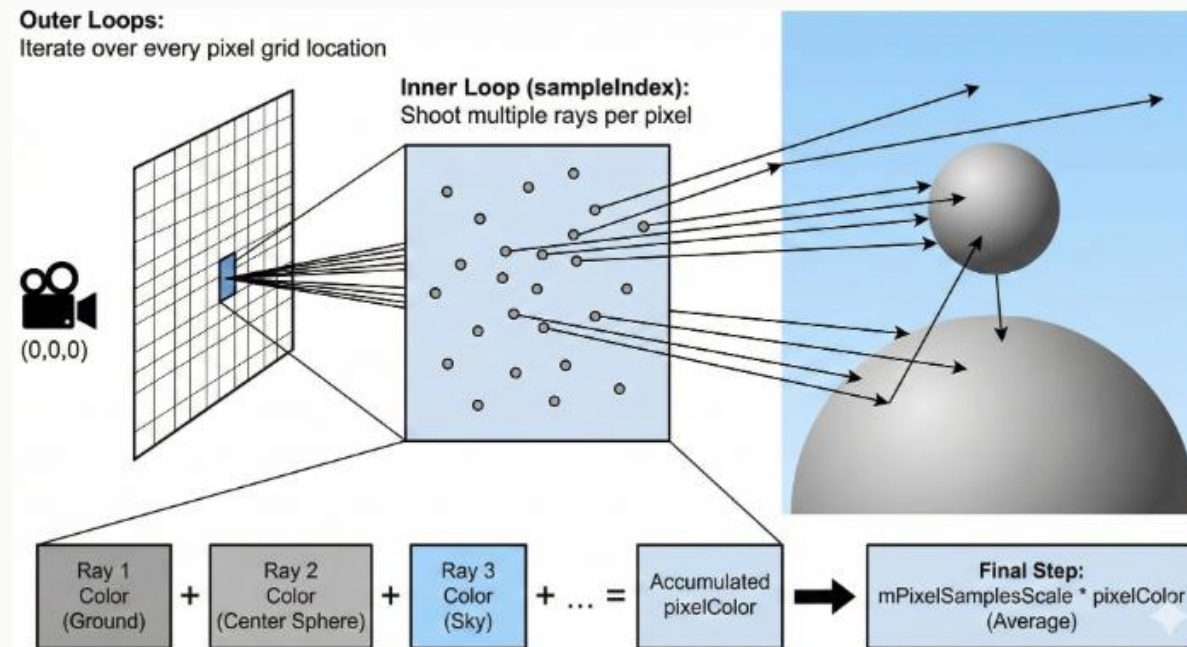
inline double RandomDouble(double minimum, double maximum)
{
    // [minimum, maximum)
    return minimum + (maximum - minimum) * RandomDouble();
}
```

색상 범위 제한: 샘플 평균값을 Clamp로 [0, 0.999] 안에 유지

색상 범위 제한

- 색상 범위 [0, 1]을 [0, 0.999]로 Clamp
- 이제 256을 곱해도 결과가 256이 되지 않음
- 목적: 샘플 색상 평균을 한 픽셀 색으로 표현
- Clamp: 평균값의 0~255 변환 시 범위 이탈 방지

픽셀 샘플링 흐름



〈그림 22〉 픽셀당 샘플 색상 누적 후 평균

Clamp: Interval 범위를 벗어난 값을 경계값으로 보정

interval.h · Clamp 추가

- 생성 시 최소 · 최대 범위 정의 후 Clamp로 제한

```
Interval(double minimum, double maximum)
    : Min(minimum)
    , Max(maximum)
{}

double Clamp(double value) const
{
    if (value < Min)
    {
        return Min;
    }
    if (value > Max)
    {
        return Max;
    }
    return value;
}
```

color.h · WriteColor

- intensity(0.000, 0.999)로 Clamp 후 256 곱셈

```
void WriteColor(std::ostream& out, const Color& pixelColor)
{
    double r = pixelColor.X();
    double g = pixelColor.Y();
    double b = pixelColor.Z();

    // [0, 1)
    static const Interval intensity(0.000, 0.999);
    int rByte = static_cast<int>(256.0 * intensity.Clamp(r));
    int gByte = static_cast<int>(256.0 * intensity.Clamp(g));
    int bByte = static_cast<int>(256.0 * intensity.Clamp(b));

    // Write Color
    out << rByte << ' ' << gByte << ' ' << bByte << '\n';
}
```

Camera 수정: 픽셀당 100회 샘플링 후 1 / 100로 평균

Camera.h 수정

- 픽셀당 샘플링 1회에서 **100회** 로 수정
- 최종 색상에 1 / 100을 곱해 평균 계산
- 광선 발사 시 [-0.5, 0.5) 범위 오프셋 사용

```
// main 시작 시, 픽셀당 샘플링 횟수 설정
camera.mSamplesPerPixel = 100;

// Camera.h, 평균을 위한 변수
mPixelSamplesScale =
    1.0 / static_cast<double>(mSamplesPerPixel);
```

GetRay · SampleSquare

```
Ray GetRay(int pixelIndex, int scanlineIndex) const
{
    Vec3 offset = SampleSquare();
    Vec3 pixelSample = mFirstPixelCenter
        + ((pixelIndex + offset.X()) * mPixelDeltaU)
        + ((scanlineIndex + offset.Y()) * mPixelDeltaV);

    Point3 rayOrigin = mCenter;
    Vec3 rayDirection = pixelSample - rayOrigin;

    return Ray(rayOrigin, rayDirection);
}

Vec3 SampleSquare() const
{
    return Vec3(RandomDouble() - 0.5,
                RandomDouble() - 0.5, 0.0);
}
```

Render: 샘플 색상을 누적한 뒤 평균으로 픽셀 색 결정

최종 렌더링

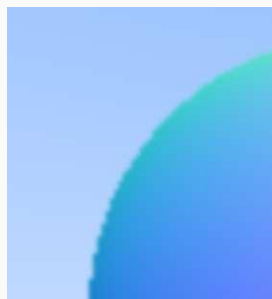
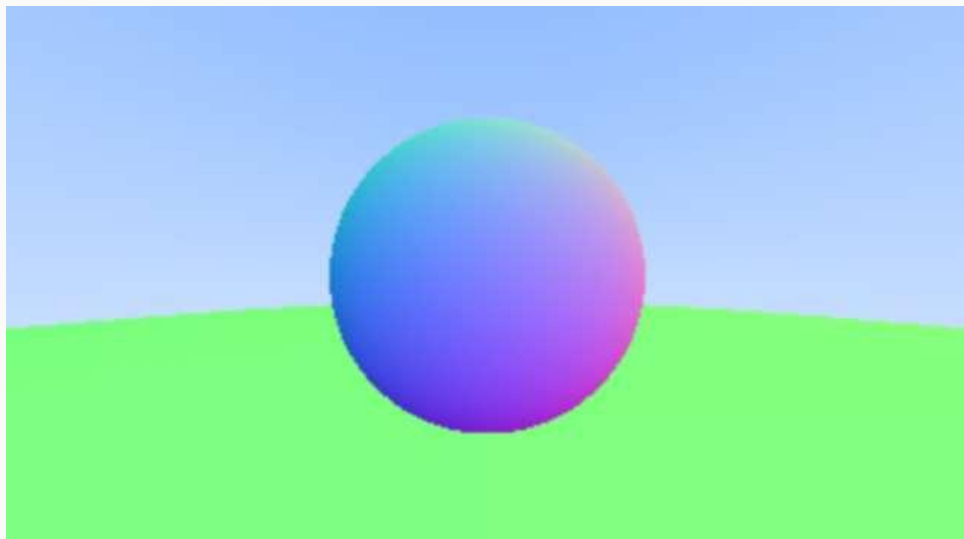
- 픽셀마다 `mSamplesPerPixel`회 `GetRay()` 호출
- 샘플별 `RayColor()`를 `pixelColor`에 누적
- `mPixelSamplesScale`을 곱해 평균 계산
- `WriteColor()`에서 Clamp로 범위 조정

Camera::Render

```
void Render(const Hittable& world)
{
    Initialize();
    ...
    for (int scanlineIndex = 0; scanlineIndex < mImageHeight;
        ++scanlineIndex)
    {
        ...
        for (int pixelIndex = 0; pixelIndex < mImageWidth;
            ++pixelIndex)
        {
            Color pixelColor(0.0, 0.0, 0.0);
            for (int sampleIndex = 0;
                sampleIndex < mSamplesPerPixel; ++sampleIndex)
            {
                Ray ray = GetRay(pixelIndex, scanlineIndex);
                pixelColor += RayColor(ray, world);
            }
            WriteColor(std::cout, mPixelSamplesScale * pixelColor);
        }
    }
}
```

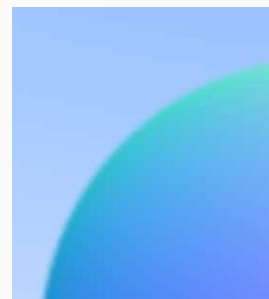
결과 비교: 수정 후 구 경계의 계단 현상 감소

수정 전



〈그림 23〉 수정 전 결과 및 확대

수정 후



〈그림 24〉 수정 후 결과 및 확대



PART 2

재질 표현

난반사, 정반사, 굴절까지

확산 재질: 들어온 빛을 무작위 방향으로 반사해 무광택 표현

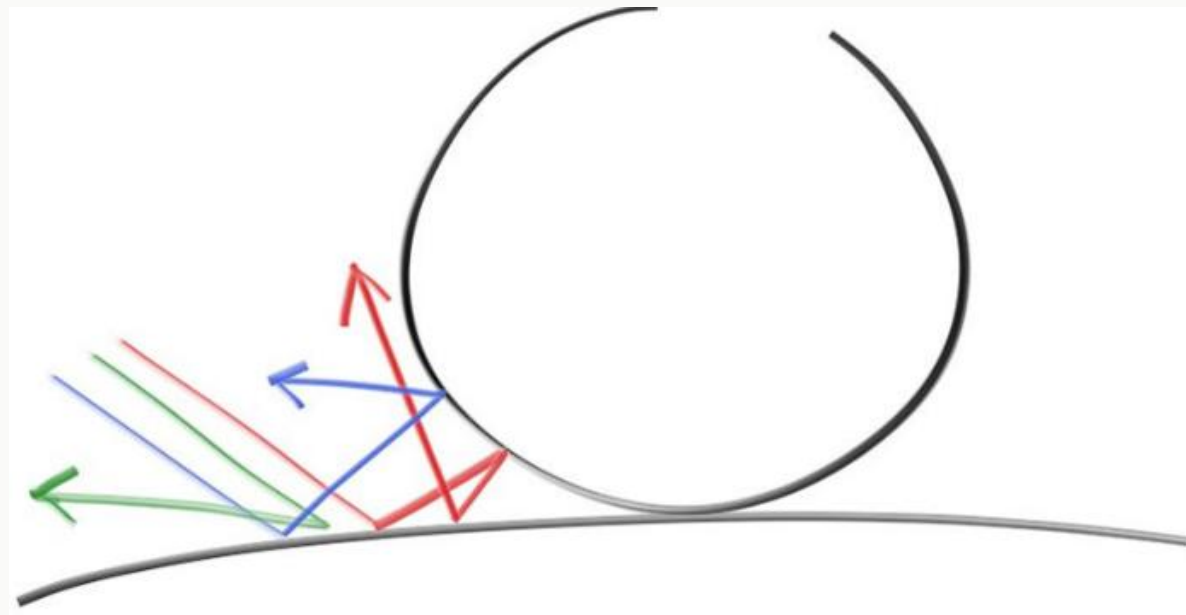
Part 2 목표

사실적으로 보이는 재질 구현

확산 재질 (Diffuse Materials)

- 스스로 빛을 내지 않음
- 들어온 빛을 무작위 방향으로 반사 → 무광택
- 들어온 빛의 일부는 흡수
- 흡수가 많은 표면일수록 반사광이 적어 어둡게 표현

확산 재질의 빛 반사

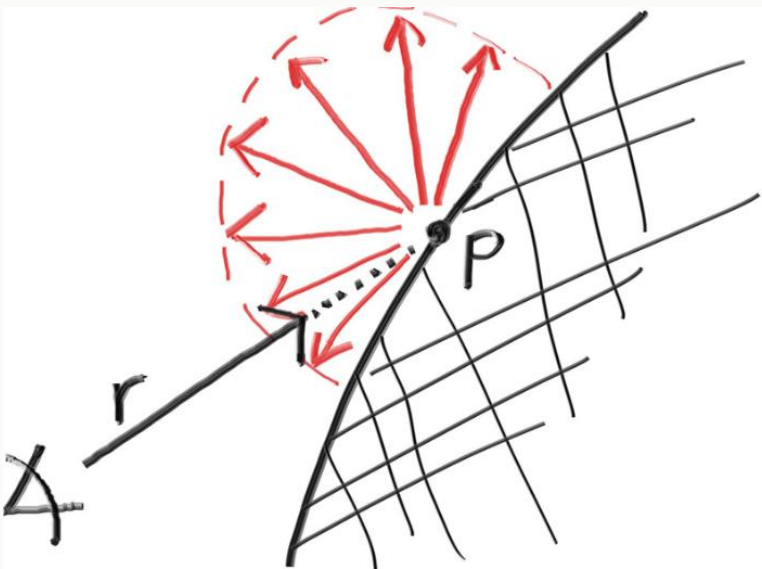


〈그림 25〉 확산 재질 표면에서 무작위 방향으로 반사되는 빛

균등 반사: 모든 방향이 같은 확률로 뵈히도록 랜덤 벡터 생성

균등한 반사

- 균등: 어느 방향이든 뵈힐 확률이 동일
- 뵈힌 방향들이 일정한 간격으로 놓인다는 의미는 아님
- 구현을 위해 랜덤 벡터 함수 추가



〈그림 26〉 균등한 무작위 반사 방향

Vec3 · Random 추가

```
// Vec3
static Vec3 Random()
{
    return Vec3(RandomDouble(), RandomDouble(),
                RandomDouble());
}

static Vec3 Random(double minimum, double maximum)
{
    return Vec3(RandomDouble(minimum, maximum),
                RandomDouble(minimum, maximum),
                RandomDouble(minimum, maximum));
}
```

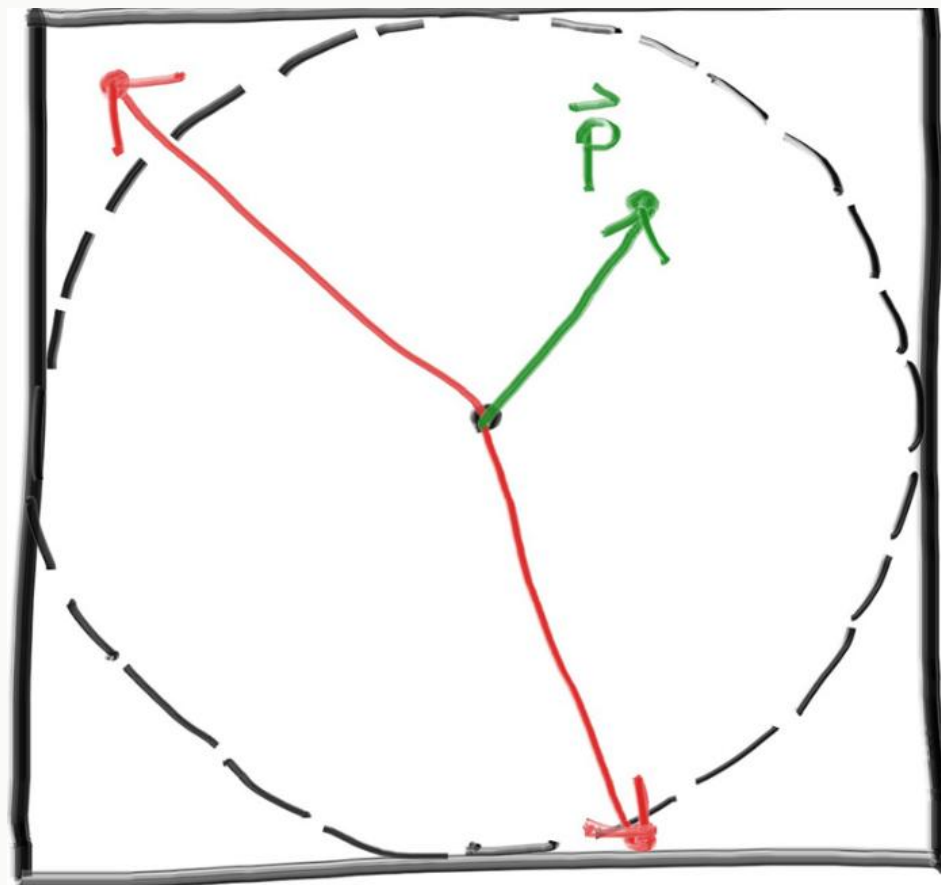
Rejection Method: 단위 구 밖의 벡터는 버리고 길이 1로 통일

단위 구를 쓰는 이유

- 가상의 단위 구에 Rejection Method 적용
- 정육면체: 대각선 방향이 더 많이 뚫힘
- 구: 어느 방향으로나 모양이 같아 균등

적용 방식

- 충돌 지점에 가상의 구가 있다고 가정
- 정육면체 기준으로 무작위로 뚫리 구를 벗어나는 벡터는 모두 버림
- 구 표면 포함 내부 벡터는 길이 1로 통일

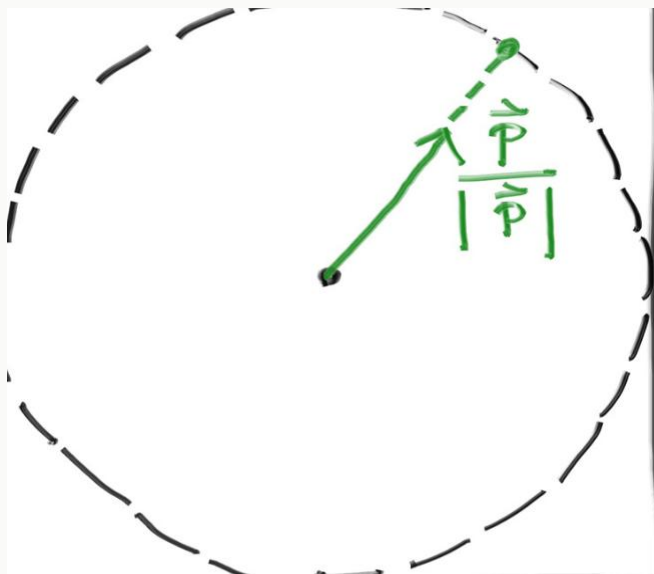


〈그림 27〉 충돌 지점의 가상 단위 구와 벡터 정규화

RandomUnitVector: 구 안의 무작위 점을 정규화해 단위 벡터 생성

랜덤 단위 벡터

- 정육면체 $[-1, 1]^3$ 안에서 무작위 점 p
- 단위 구 밖이면 ($길이^2 > 1$) 다시 뽑음
- 단위 구 안이면 길이로 나눠 정규화



〈그림 28〉 단위 구 안의 무작위 점 선택

RandomUnitVector

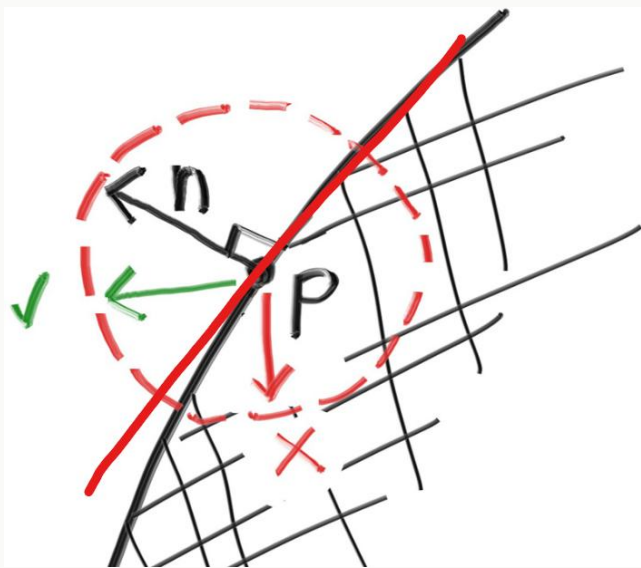
```
inline Vector3 RandomUnitVector()
{
    while (true)
    {
        Vector3 p = Vector3::Random(-1.0, 1.0);
        double lengthSquared = p.LengthSquared();

        if (1e-160 < lengthSquared && lengthSquared <= 1.0)
        {
            return p / std::sqrt(lengthSquared);
        }
    }
}
```

RandomOnHemisphere: 법선 반대편 벡터는 뒤집어 반구 안으로

반구 판별

- 랜덤 벡터를 표면 법선과 비교
- 90도를 넘는 반대편 벡터는 뒤집어서 사용



〈그림 29〉 법선 기준 반구와 반대편 벡터 뒤집기

RandomOnHemisphere

```
inline Vector3 RandomOnHemisphere(const Vector3& normal)
{
    Vector3 unitSphereDirection = RandomUnitVector();

    if (Dot(unitSphereDirection, normal) > 0.0)
    {
        return unitSphereDirection;
    }

    return -unitSphereDirection;
}
```

결과 1- 광선 반사량: 충돌마다 반사 광선 색의 50%만 유지

반사율

- 반사율 100% → 흰색, 0% → 검은색
- 먼저 50%로 설정해 회색 구 결과 예상

RayColor 동작

- 맞았을 때: 점 P에서 반구 방향 새 광선
- 새 광선의 RayColor 결과를 0.5배 반환
- 안 맞았을 때: 하늘색 반환 (재귀 종료)

색 결정 방식 변경

- 법선 좌표 기반 → 재질의 빛 반사 기반

Camera::RayColor

```
Color RayColor(const Ray& ray, const Hittable& world)
{
    HitRecord hitRecord;
    if (world.Hit(ray, Interval(0.0, Infinity), hitRecord))
    {
        Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
        return 0.5 * RayColor(Ray(hitRecord.P, direction),
                               world);
    }

    Vec3 unitDirection = UnitVector(ray.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0);
}
```

결과 2- 재귀 깊이 제한: depth로 무한 반사와 스택 오버플로우 방지

무한 반사 문제

- 배경에 닿을 때까지 횟수 제한 없이 반사
- 스택 오버플로우 방지 → 최대 재귀 깊이
- 파라미터에 **depth** 추가
- MaxDepth부터 호출, depth > 0이면 반사 가능

Render() · mMaxDepth 전달

```
// Render()
for (int sampleIndex = 0;
    sampleIndex < mSamplesPerPixel; ++sampleIndex)
{
    Ray ray = GetRay(pixelIndex, scanlineIndex);
    pixelColor += RayColor(ray, mMaxDepth, world);
}
```

Camera::RayColor · depth 추가

```
Color RayColor(const Ray& ray, int depth,
               const Hittable& world)
{
    if (depth <= 0)
    {
        return Color(0.0, 0.0, 0.0);
    }
    HitRecord hitRecord;
    if (world.Hit(ray, Interval(0.0, Infinity), hitRecord))
    {
        Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
        return 0.5 * RayColor(Ray(hitRecord.P, direction),
                              depth - 1, world);
    }
    Vec3 unitDirection = UnitVector(ray.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0); // background
}
```

결과 3- Shadow Acne: 0.001 미만의 가까운 충돌 무시

Shadow Acne

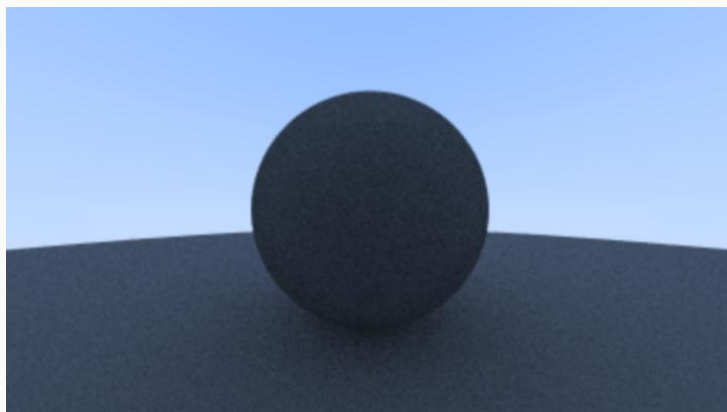
- 부동소수점 오류로 표면과 다시 충돌
- 매우 가까운 충돌은 무시해 방지
- 최소 충돌 가능 범위: 0.001

Camera::RayColor · 최소 충돌 범위

```
// RayColor()
if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
{
    Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
    return 0.5 * RayColor(Ray(hitRecord.P, direction),
        depth - 1, world);
}
```

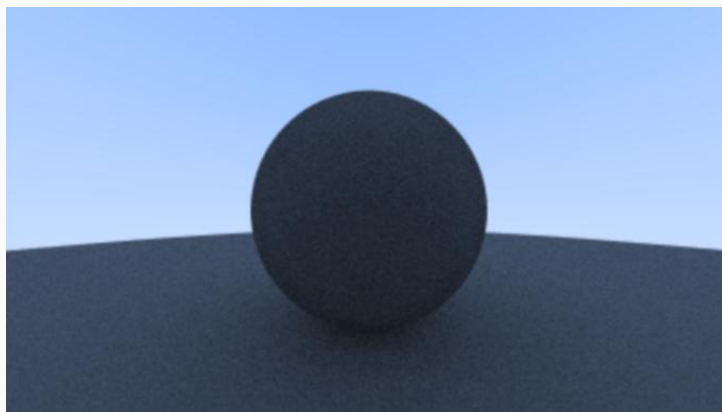
결과 비교: 재귀 깊이 제한과 최소 충돌 범위 적용

반사율 50% · 재귀 제한 없음



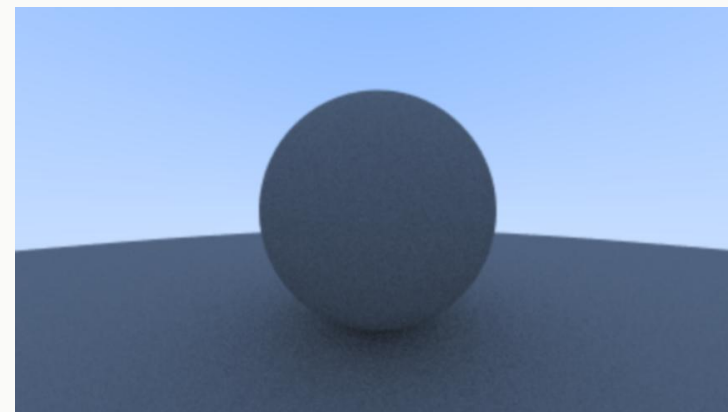
〈그림 30〉 첫 확산 구 결과

재귀 깊이 제한 · depth



〈그림 31〉 재귀 깊이 제한 적용

최소 충돌 범위 · 0.001

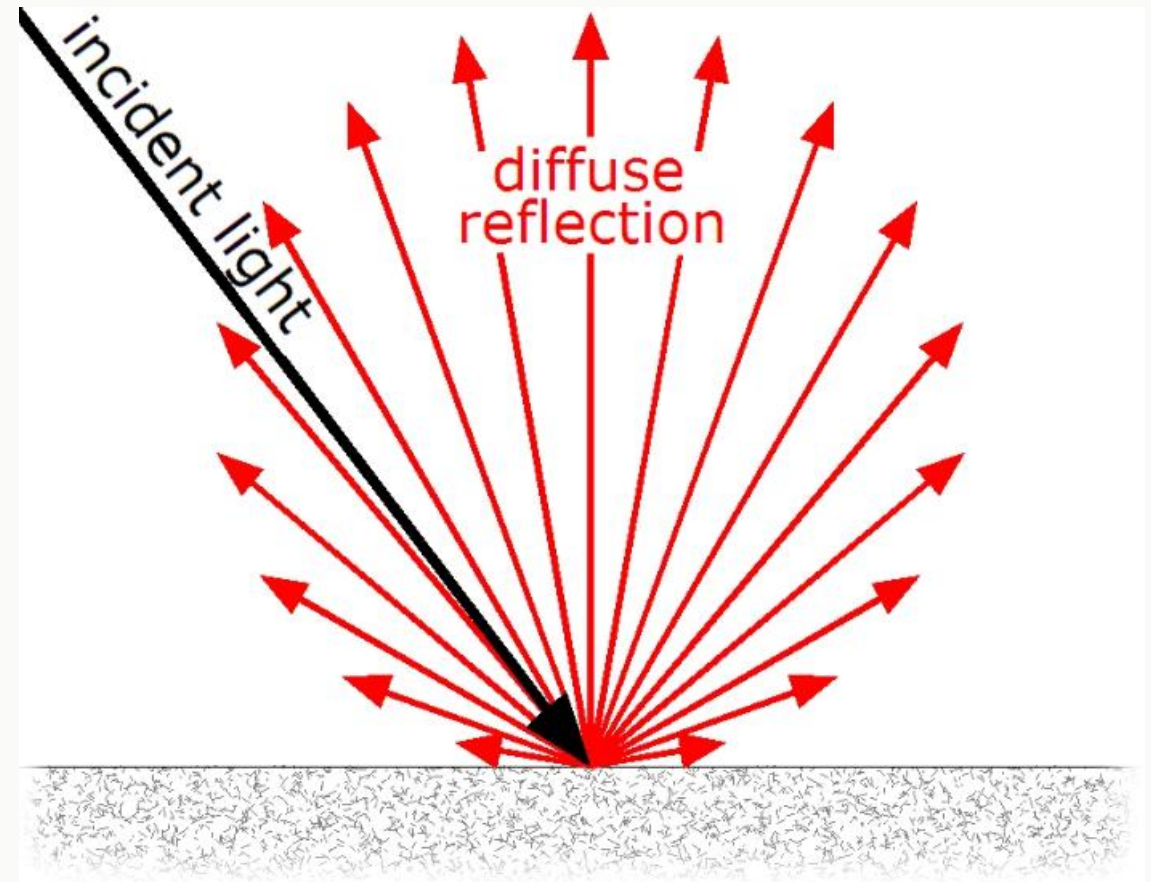


〈그림 32〉 최소 충돌 범위 적용

Lambertian Reflection: $\cos \theta$ 에 비례해 법선 방향으로 산란 방식 사용

Lambertian 분포

- 반구 균등 반사: 부드러운 확산 모델
- 실제 확산 물체: Lambertian 분포가 더 정확
- 반사 광선을 $\cos \theta$ 에 비례해 산란
- θ : 반사 광선과 표면 법선 사이 각도
- 법선에 가까울수록 산란 가능성 높음



〈그림 33〉 법선 방향으로 이동한 가상의 구 (중심 $P + n$)

Lambertian 구현: 법선 + 랜덤 단위 벡터로 반사 방향 결정

반사 방향 변경

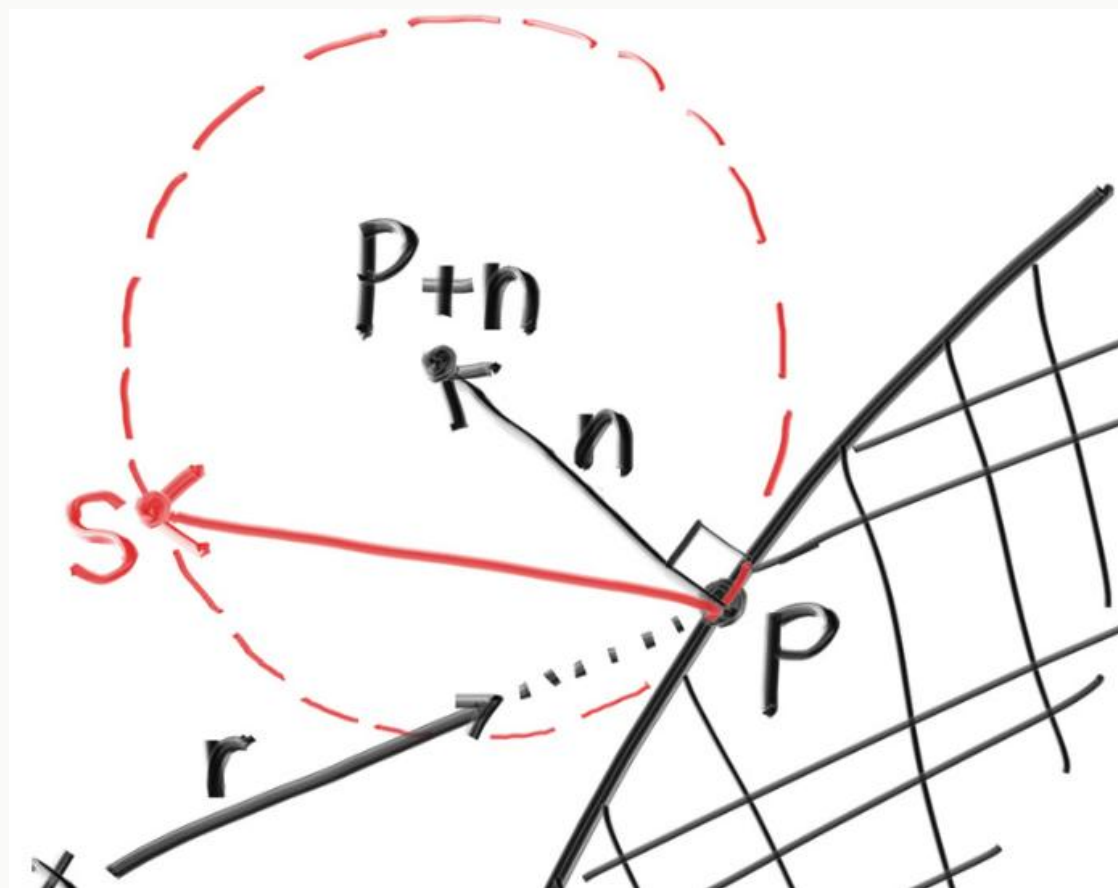
- 반사 방향: 법선 + 랜덤 단위 벡터
- 분포가 법선 방향으로 몰림

Camera::RayColor

```
// Camera::RayColor()  
Vec3 direction = hitRecord.Normal + RandomUnitVector();  
// RandomOnHemisphere 대체
```

가상의 구 이동

- 가상의 구를 법선 방향으로 1만큼 이동
- 구 중심: P가 아닌 $P + n$



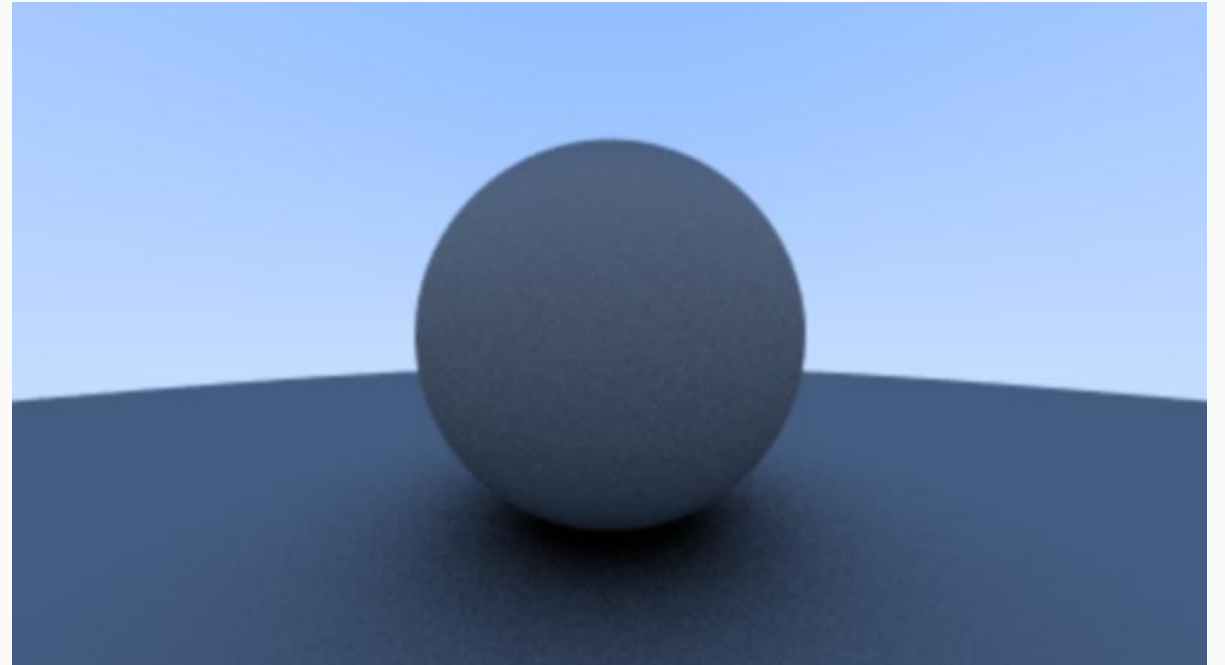
〈그림 34〉 Lambertian 분포에 따른 반사 방향

Lambertian 결과: 더 뚜렷해진 그림자와 짙어진 하늘색

변화

- 그림자가 더 뚜렷
- 두 구 모두 하늘 배경색을 더 짙

Lambertian 반사 결과



〈그림 35〉 Lambertian 반사 적용 결과

선형 공간 데이터를 그대로 넘겨 어둡게 표시되는 문제

반사 결과

- 50%가 중간 지점이기엔 너무 어두움

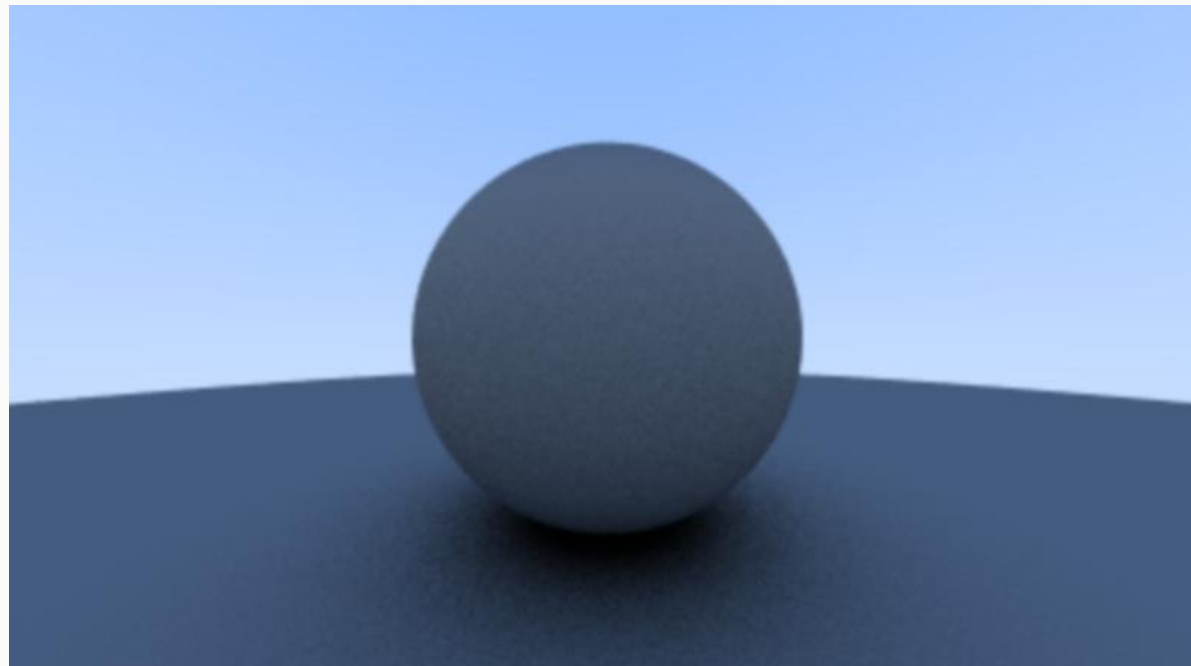
원인

- 이미지 뷰어에서는 감마 보정되었다고 가정
- 감마 공간을 기대하지만 선형 공간 전달하여 어둡게 표시

*선형 공간: 빛의 세기에 비례하는 값

*감마 공간: 거듭제곱으로 변환된 값

Lambertian 반사 결과



〈그림 35〉 Lambertian 반사 적용 결과

LinearToGamma: 제공근으로 선형 공간을 감마 공간으로 변환

감마 2 근사

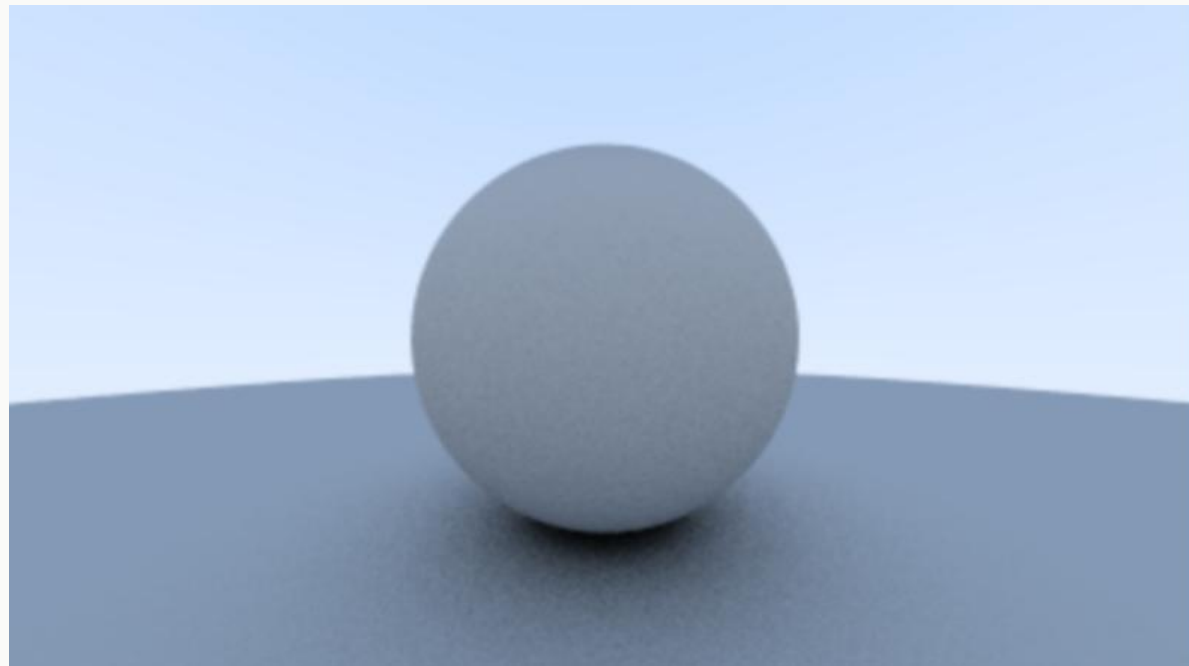
- 간단한 근사로 감마 2 사용
- 감마 → 선형 변환 시 거듭제곱 2
- 선형 → 감마: 제공근으로 색상 변환

LinearToGamma

```
inline double LinearToGamma(double linearComponent)
{
    if (linearComponent > 0.0)
    {
        return std::sqrt(linearComponent);
    }

    return 0.0;
}
```

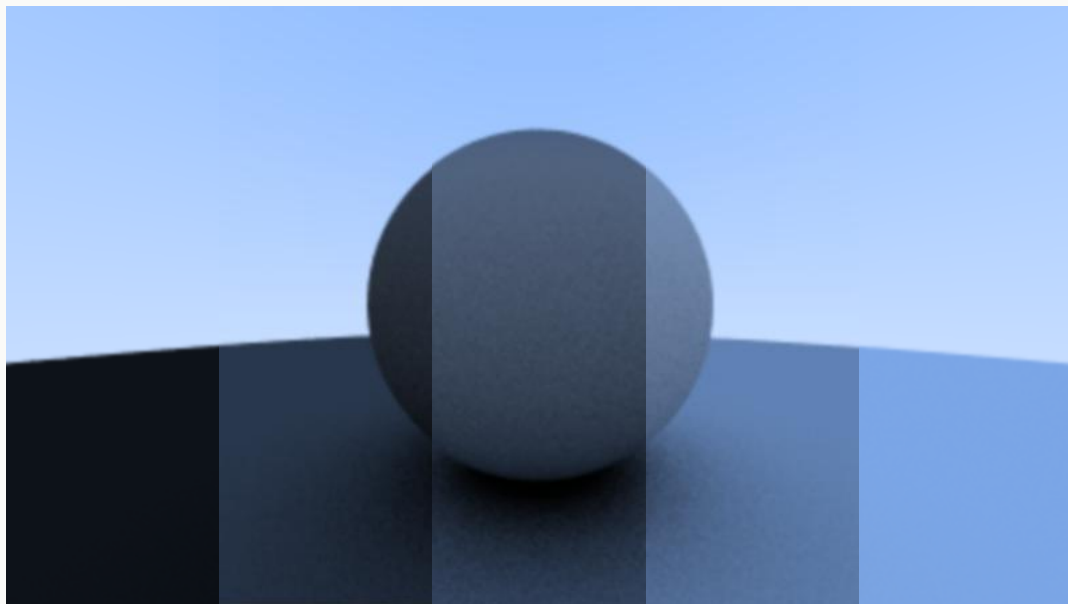
수정 후 반사 결과



〈그림 36〉 감마 보정 후 반사율 50% 결과

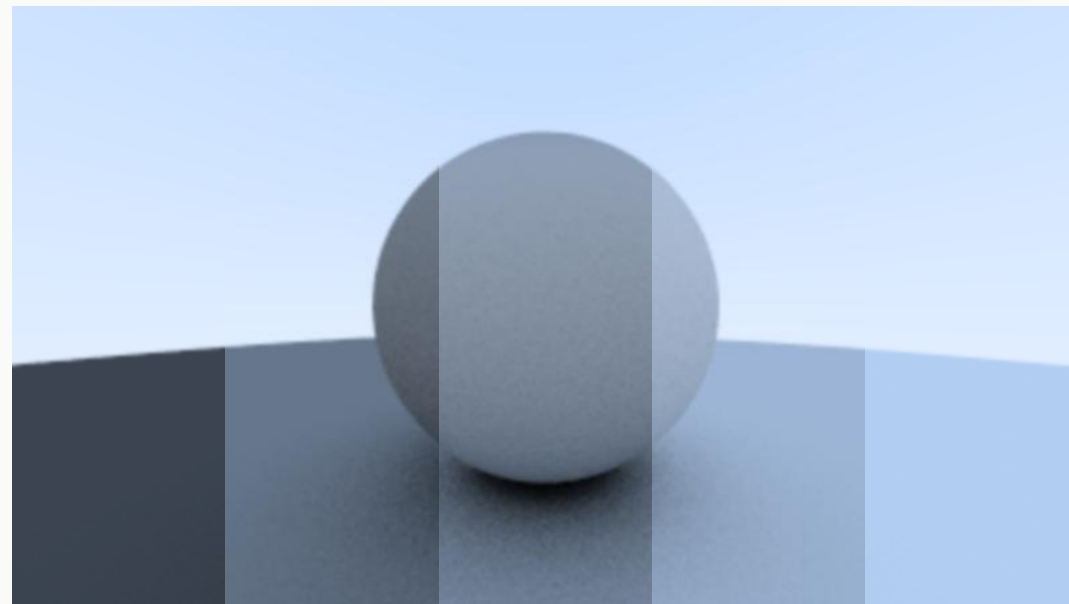
결과 비교: 감마 보정 후 더 밝아진 렌더링 (10, 30, 50, 70, 90% 비교)

감마 보정 전



〈그림 37〉 감마 보정 전 렌더링 결과

감마 보정 후 · LinearToGamma



〈그림 38〉 감마 보정 후 더 밝아진 결과

Material: 추상 클래스로 객체마다 다른 재질 부여

재질 추상화

- 서로 다른 객체가 서로 다른 재질을 갖도록 함
- 추상 클래스 Material 정의 후 각 재질이 상속

Material.h

```
// Material.h
#ifndef MATERIAL_H
#define MATERIAL_H

#include "hittable.h"
#include "rtweekend.h"

class Material
{
public:
    virtual ~Material() = default;

    virtual bool Scatter(const Ray& rayIn,
                        const HitRecord& hitRecord, Color& attenuation,
                        Ray& scattered) const
    {
        return false;
    }
};
#endif
```

HitRecord · Sphere: 충돌 기록과 구에 재질 포인터 추가

HitRecord · 재질 포인터

- 충돌 기록 HitRecord가 **Material** 포인터 보유
- **전방 선언**으로 순환 참조 방지
- 포인터만 가지므로 클래스 세부 사항 불필요

```
class Material;

class HitRecord
{
public:
    void SetFaceNormal(const Ray& ray,
                      const Vec3& outwardNormal) { ... }
public:
    Point3 P;
    Vec3 Normal;
    double T;
    bool bFrontFace;
    std::shared_ptr<Material> material;
};
```

Sphere · 재질 정보 보유

- 생성 시 재질을 받아 **mMaterial**에 저장
- 충돌 시 hitRecord.material에 재질 전달

```
Sphere(const Point3& center, double radius,
       const std::shared_ptr<Material>& material)
    : mCenter(center), mRadius(std::fmax(0.0, radius)),
      mMaterial(material)
{
}

bool Hit(const Ray& ray, const Interval& rayT,
        HitRecord& hitRecord) const override
{
    ...
    hitRecord.material = mMaterial;
    return true;
}

private:
    std::shared_ptr<Material> mMaterial;
```

Lambertian: albedo로 R · G · B 채널별 반사율 지정

albedo

- 라틴어로 "백색도"
- 여기서는 반사율: 들어온 빛 중 반사되는 비율
- 재질 색에 따라 R, G, B별로 다름
- 앞 장치럼 무채색 통일 안 함
- 유리처럼 입사 방향에 따라 달라지는 경우도 존재

확산 재질을 클래스로 이동

- 앞 장에서 구현한 확산 재질을 이동
- 항상 산란하도록 설정 → **true** 반환

Material.h · Lambertian

```
class Lambertian : public Material
{
public:
    explicit Lambertian(const Color& albedo)
        : mAlbedo(albedo)
    {
    }
    bool Scatter(const Ray& rayIn,
                 const HitRecord& hitRecord, Color& attenuation,
                 Ray& scattered) const override
    {
        Vec3 scatterDirection = hitRecord.Normal
            + RandomUnitVector();
        scattered = Ray(hitRecord.P, scatterDirection);
        attenuation = mAlbedo;
        return true;
    }
private:
    Color mAlbedo;
};
```

NearZero: 0에 가까운 산란 방향은 법선 방향으로 대체

방향이 0인 광선 문제

- 랜덤 단위 벡터가 법선과 정확히 반대인 경우
- 두 벡터의 합 = 0 → **방향이 0인 광선**
=> 무한대와 NaN 같은 문제 발생
- 0에 매우 가까우면 **법선 방향으로 대체**

vec3.h · NearZero

```
bool NearZero() const
{
    double threshold = 1e-8;
    // Return true if the vector is too close
    // to zero in all dimensions
    return (std::fabs(E[0]) < threshold)
        && (std::fabs(E[1]) < threshold)
        && (std::fabs(E[2]) < threshold);
}
```

Material.h · Lambertian::Scatter

```
bool Scatter(const Ray& rayIn,
             const HitRecord& hitRecord, Color& attenuation,
             Ray& scattered) const override
{
    Vec3 scatterDirection = hitRecord.Normal
        + RandomUnitVector();

    if (scatterDirection.NearZero())
    {
        scatterDirection = hitRecord.Normal;
    }

    scattered = Ray(hitRecord.P, scatterDirection);
    attenuation = mAlbedo;
    return true;
}
```

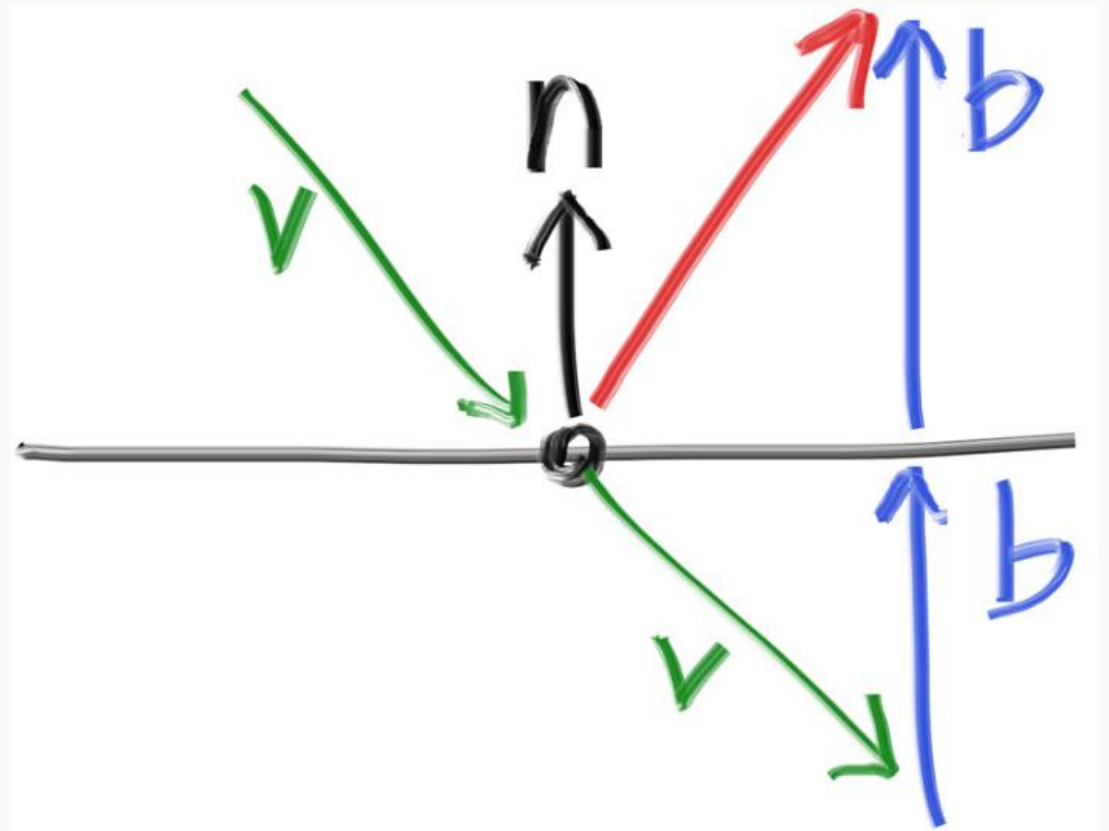
거울 반사: 반사 벡터 $\mathbf{l} = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$

광택 있는 금속의 광선 반사

- $\mathbf{v}$: 들어오는 방향, $\mathbf{n}$: 표면 외부 방향
- $\mathbf{v} \cdot \mathbf{n}$ 은 스칼라 $\rightarrow$ $\mathbf{n}$ 을 곱해 $\mathbf{n}$ 방향 부여
- $\mathbf{v} \cdot \mathbf{n} < 0 \rightarrow -$ 부호로 표면 바깥 방향 $\mathbf{b}$
- $\mathbf{v}$ 에서 $\mathbf{b}$ 를 2번 더해서 반사 벡터 계산

vec3.h · Reflect

```
// vec3.h
inline Vector3 Reflect(const Vector3& v,
    const Vector3& n)
{
    return  $\mathbf{v} - 2.0 * \text{Dot}(\mathbf{v}, \mathbf{n}) * \mathbf{n}$ ;
}
```



〈그림 39〉 금속 거울에서의 광선 반사

Metal: 반사 공식으로 구한 방향을 산란 광선으로 사용

Metal 클래스

- **Material**을 상속받는 Metal 클래스 추가
- **Reflect** 공식으로 반사 방향 계산
- 충돌 지점 P에서 반사 방향으로 새 광선

Material.h · Metal

```
class Metal : public Material
{
public:
    explicit Metal(const Color& albedo)
        : mAlbedo(albedo)
    {
    }

    bool Scatter(const Ray& rayIn,
                 const HitRecord& hitRecord, Color& attenuation,
                 Ray& scattered) const override
    {
        Vec3 reflected = Reflect(rayIn.Direction(),
                                   hitRecord.Normal);
        scattered = Ray(hitRecord.P, reflected);
        attenuation = mAlbedo;
        return true;
    }
private:
    Color mAlbedo;
};
```

RayColor: 충돌한 물체 재질의 Scatter만 호출

Camera 수정

- 추가된 기능을 올바르게 호출하기 위한 수정
- 각 재질 클래스가 가상 함수 **Scatter** 구현
- RayColor는 **Scatter만 호출**

Scatter 매개변수

매개변수	방향	뜻
rayIn	입력	들어온 광선 · 금속 반사에 사용
hitRecord	입력	충돌 지점 P, 법선, 앞면 여부
attenuation	출력	튀긴 광선에 채널별로 곱할 감쇠 색
scattered	출력	튀겨 나갈 새 광선
반환값 bool	출력	true = 산란, false = 흡수

Camera::RayColor

```
Color RayColor(const Ray& ray, int depth,
               const Hittable& world)
{
    if (depth <= 0)
    {
        return Color(0.0, 0.0, 0.0);
    }
    HitRecord hitRecord;
    if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
    {
        Ray scattered;
        Color attenuation;
        if (hitRecord.material->Scatter(ray, hitRecord,
                                       attenuation, scattered))
        {
            return attenuation * RayColor(scattered, depth - 1, world);
        }
        return Color(0.0, 0.0, 0.0);
    }
    Vec3 unitDirection = UnitVector(ray.Direction());
    double a = 0.5 * (unitDirection.Y() + 1.0);
    return (1.0 - a) * Color(1.0, 1.0, 1.0)
        + a * Color(0.5, 0.7, 1.0); // background
}
```

World 구성 및 결과: 재질을 먼저 만든 뒤 구에 할당

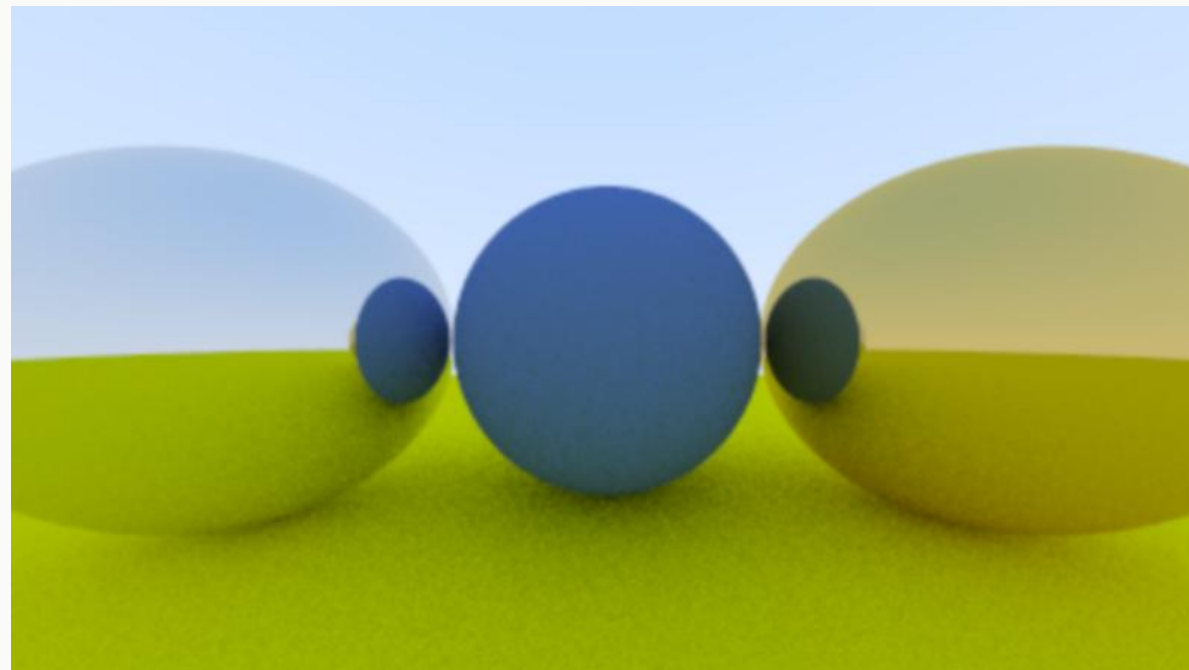
main · 재질과 구 추가

- 충돌 가능한 물체를 월드에 추가
- 각 재질 생성 후 구에 해당 재질 사용

```
// main()
auto materialGround =
    std::make_shared<Lambertian>(Color(0.8, 0.8, 0.0));
auto materialCenter =
    std::make_shared<Lambertian>(Color(0.1, 0.2, 0.5));
auto materialLeft =
    std::make_shared<Metal>(Color(0.8, 0.8, 0.8));
auto materialRight =
    std::make_shared<Metal>(Color(0.8, 0.6, 0.2));

world.Add(std::make_shared<Sphere>(
    Point3(0.0, -100.5, -1.0), 100.0, materialGround));
world.Add(std::make_shared<Sphere>(
    Point3(0.0, 0.0, -1.2), 0.5, materialCenter));
world.Add(std::make_shared<Sphere>(
    Point3(-1.0, 0.0, -1.0), 0.5, materialLeft));
world.Add(std::make_shared<Sphere>(
    Point3(1.0, 0.0, -1.0), 0.5, materialRight));
```

결과 - 광택 있는 금속



〈그림 40〉 광택 있는 금속 구 렌더링 결과

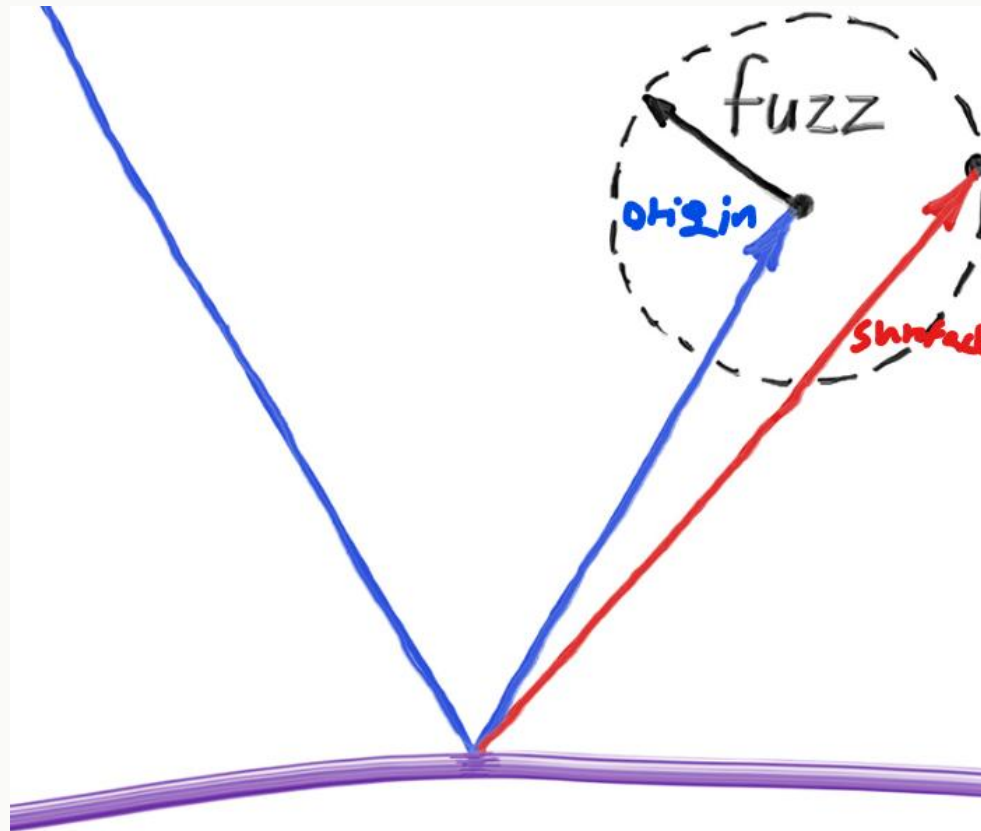
Fuzzy Reflection: 퍼지 구 위 무작위 점으로 반사 방향 변경

부드러운 반사

- 실제 금속 대부분은 표면이 미세하게 거칠
- 예: 브러시드 스테인리스, 숟가락 뒷면
- 비친 모습이 뿌옇게 번지는 현상 흉내

구현 방식

- 반사 방향을 퍼지 구 표면의 무작위 점으로 변경
- 반사 벡터가 처음부터 다른 방향으로 뻗음
- 퍼지 구 중심: 반사 방향으로 거리 1인 지점
- 퍼지 구 반지름 = 퍼지 계수



〈그림 41〉 퍼지 구를 이용한 반사 방향 선택

기존 반사 벡터에 'fuzz × 랜덤 단위 벡터' 합침

퍼지 계수 (Fuzz Factor)

- 원래 끝점 중심의 가상 구에서 무작위 점 선택
- 이를 퍼지 계수로 스케일링
- 퍼지 구가 클수록 흐릿한 반사
=> fuzz 계수는 최대 1까지만 허용

Metal · 생성자

```
explicit Metal(const Color& albedo, double fuzz)
    : mAlbedo(albedo),
      mFuzz(fuzz < 1.0 ? fuzz : 1.0)
{
}
```

Metal::Scatter

```
bool Scatter(const Ray& rayIn,
             const HitRecord& hitRecord, Color& attenuation,
             Ray& scattered) const override
{
    Vec3 reflected = Reflect(rayIn.Direction(),
                             hitRecord.Normal);

    // fuzz 구의 중심까지 벡터 + 중심에서 표면까지 벡터
    reflected = UnitVector(reflected)
        + (mFuzz * RandomUnitVector());

    scattered = Ray(hitRecord.P, reflected);
    attenuation = mAlbedo;
    return (Dot(scattered.Direction(),
               hitRecord.Normal) > 0);
}
```

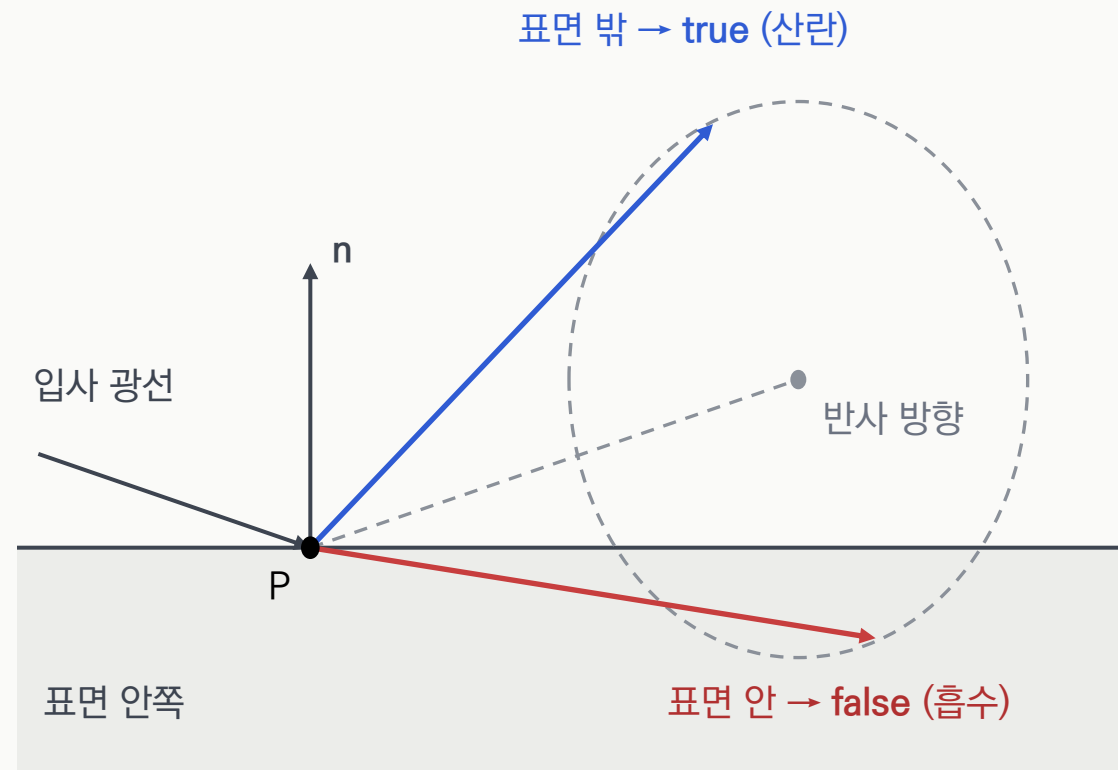
스치는 반사: 표면 안으로 파고드는 산란은 false 반환

스치는 반사의 문제

- 스치는 경우의 반사에서 문제 발생 가능
- 퍼지 구 일부가 표면 아래로 들어감
- 표면 안으로 파고드는 반사는 false 반환
=> false 반환 → 광선 흡수

Metal::Scatter · 반환값

```
return (Dot(scattered.Direction(), hitRecord.Normal) > 0);
```



〈그림 42〉 스치는 반사에서 표면 안으로 파고드는 산란

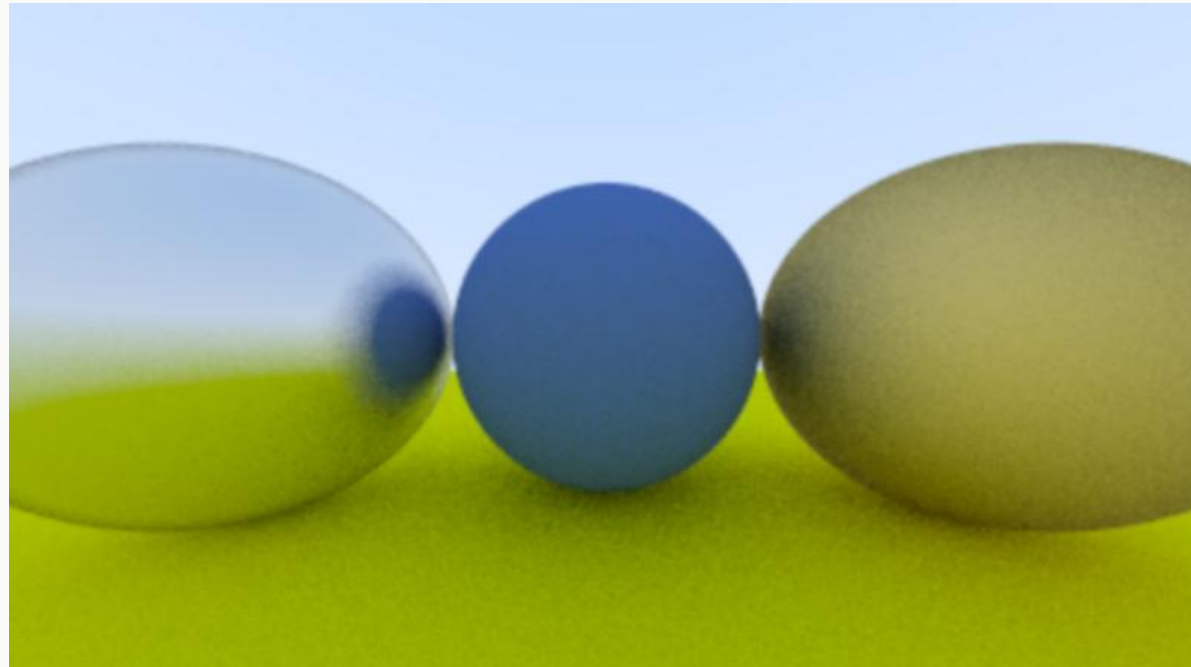
결과: fuzz 계수가 클수록 더 흐릿한 금속 구

main · fuzz 계수 지정

- Fuzz 계수가 높을수록 흐릿한 금속
- 왼쪽 구 0.3, 오른쪽 구 1.0

```
auto materialLeft = std::make_shared<Metal>(
    Color(0.8, 0.8, 0.8), 0.3);
auto materialRight = std::make_shared<Metal>(
    Color(0.8, 0.6, 0.2), 1.0);
```

결과 - Fuzzy Reflection



〈그림 43〉 fuzz 계수 적용 후 흐릿한 금속 구 결과

유전체: 투명 재질에서 광선은 반사와 굴절로 분리

유전체 (Dielectrics)

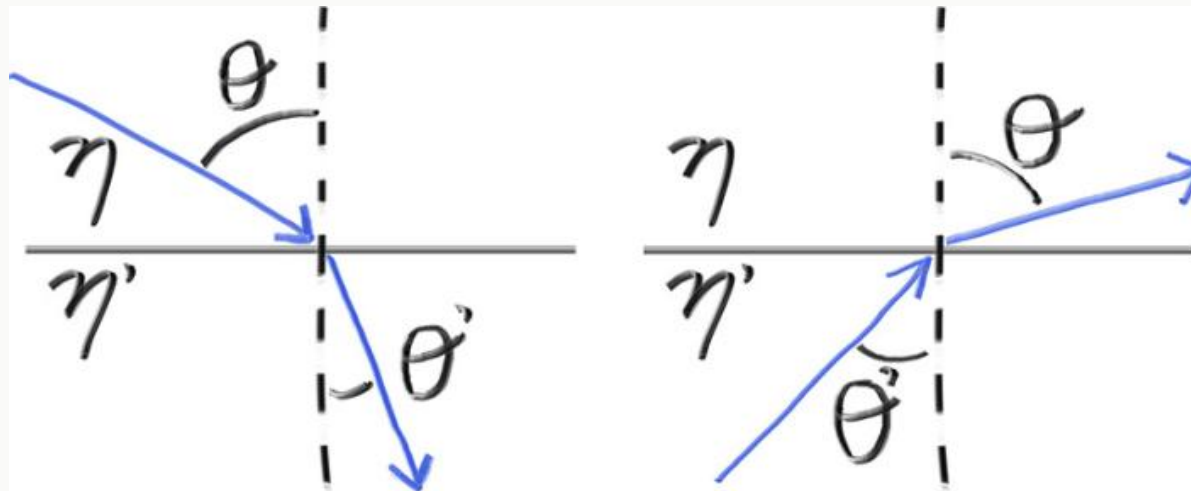
- 물, 유리, 다이아몬드 같은 투명 재질
- 광선이 닿으면 반사 광선과 굴절(투과) 광선으로 분리
- 굴절 광선은 물질의 굴절률로 결정

스넬의 법칙 (Snell's Law)

$$\eta \sin \theta = \eta' \sin \theta'$$

- 굴절은 스넬의 법칙으로 설명, 동일 환경이면 항상 성립
- $\cos$ = 법선 방향 길이, $\sin$ = 표면 방향 길이
- R: 들어오는 광선 방향, R': 굴절되어 나가는 광선 방향

반사와 굴절



〈그림 44〉 유전체 경계에서의 굴절

스넬의 법칙 변형: 굴절각은 굴절률 비율로 결정

굴절각 구하기

- 스넬의 법칙으로 $\sin \theta'$ 를 먼저 구함

$$\sin \theta' = (n / n') \sin \theta$$

$n / n' < 1$ (공기 $\rightarrow$ 유리)

$\theta' < \theta$

법선 쪽으로 꺾임

$n / n' > 1$ (유리 $\rightarrow$ 공기)

$\theta' > \theta$

법선에서 멀어지게 꺾임

필요한 것: 굴절 방향 벡터 R'

- 코드에 필요한 건 각도가 아닌 **방향 벡터**
- $R' \perp$ 길이 = $\sin \theta' \rightarrow$ 스넬의 법칙으로 이미 알
- $R' \parallel$ 길이 = $\cos \theta' \rightarrow$ 전체 길이 1이 되도록 구함

벡터 분해

$$R = (R \cdot n) n + \text{표면 성분}$$

$$R' = R' \perp + R' \parallel$$

$(R \cdot n) n$: 법선 성분

굴절 벡터 유도: $\cos \theta \rightarrow$ 표면 성분 $\rightarrow$ 법선 성분 순서로 계산

법선 성분 크기: $\cos \theta$

$$\cos \theta = -\mathbf{R} \cdot \mathbf{n}$$

- R은 안쪽, n은 바깥 방향 $\rightarrow$ 내적이 음수
- '-'를 붙여 양수 $\cos \theta$ 획득

굴절 방향의 표면 성분: $\mathbf{R}'_{\perp}$

$$\mathbf{R} - (\mathbf{R} \cdot \mathbf{n}) \mathbf{n} = \mathbf{R} + \cos \theta \mathbf{n}$$

$$\mathbf{R}'_{\perp} = (\eta / \eta')(\mathbf{R} + \cos \theta \mathbf{n})$$

- R에서 법선 성분을 빼면 표면 성분
- R의 표면 성분을 η / η' 배

굴절 방향의 법선 성분: $\mathbf{R}'_{\parallel}$

$$1 = |\mathbf{R}'_{\perp}|^2 + |\mathbf{R}'_{\parallel}|^2$$

$$|\mathbf{R}'_{\parallel}| = \sqrt{1 - |\mathbf{R}'_{\perp}|^2}$$

- 전체 길이 1을 이용해 법선 성분 계산

Refract: 표면 성분과 법선 성분을 더해 굴절 방향 계산

항상 굴절하는 유리 구

- 기존의 좌측 구를 유리 재질로 지정

```
auto materialLeft = std::make_shared<Dielectric>(1.50);
```

Refract · 굴절 방향 계산

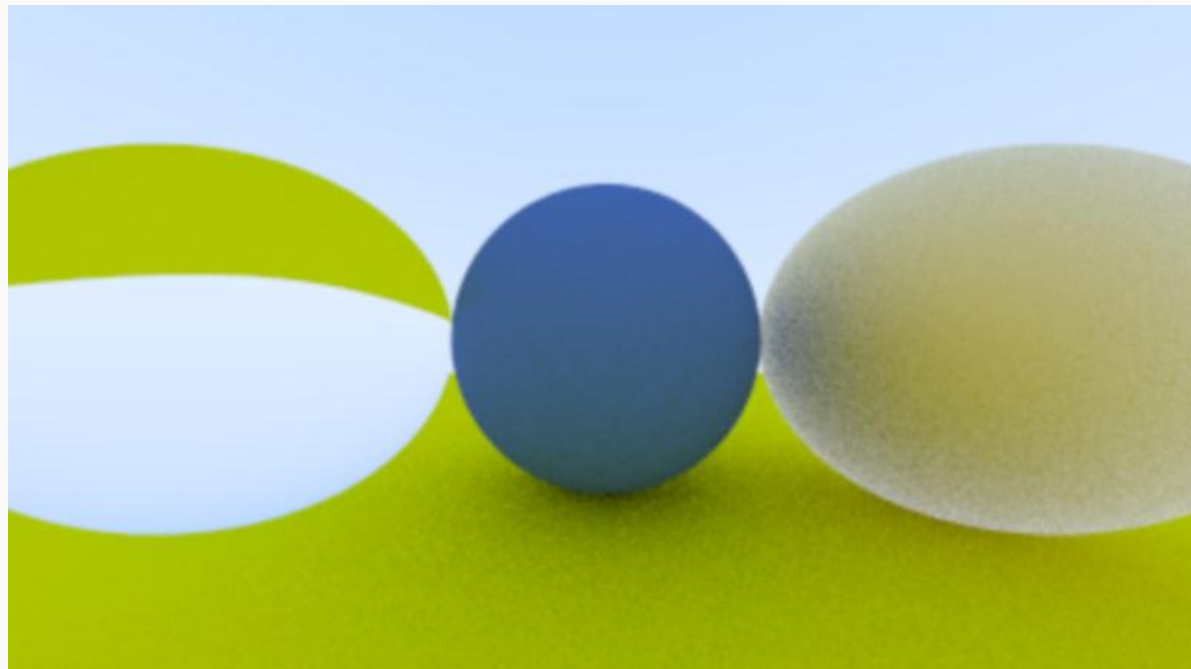
```
double cosTheta = std::fmin(Dot(-uv, n), 1.0);

Vec3 refractPerpendicular =
    etaInOverEtaOut * (uv + cosTheta * n);

Vec3 refractParallel = -std::sqrt(std::fabs(
    1.0 - refractPerpendicular.LengthSquared())) * n;

return refractPerpendicular + refractParallel;
```

결과 - 항상 굴절하는 유리 구 ($\eta = 1.5$)



〈그림 45〉 굴절률 1.5 유리 구 렌더링 결과

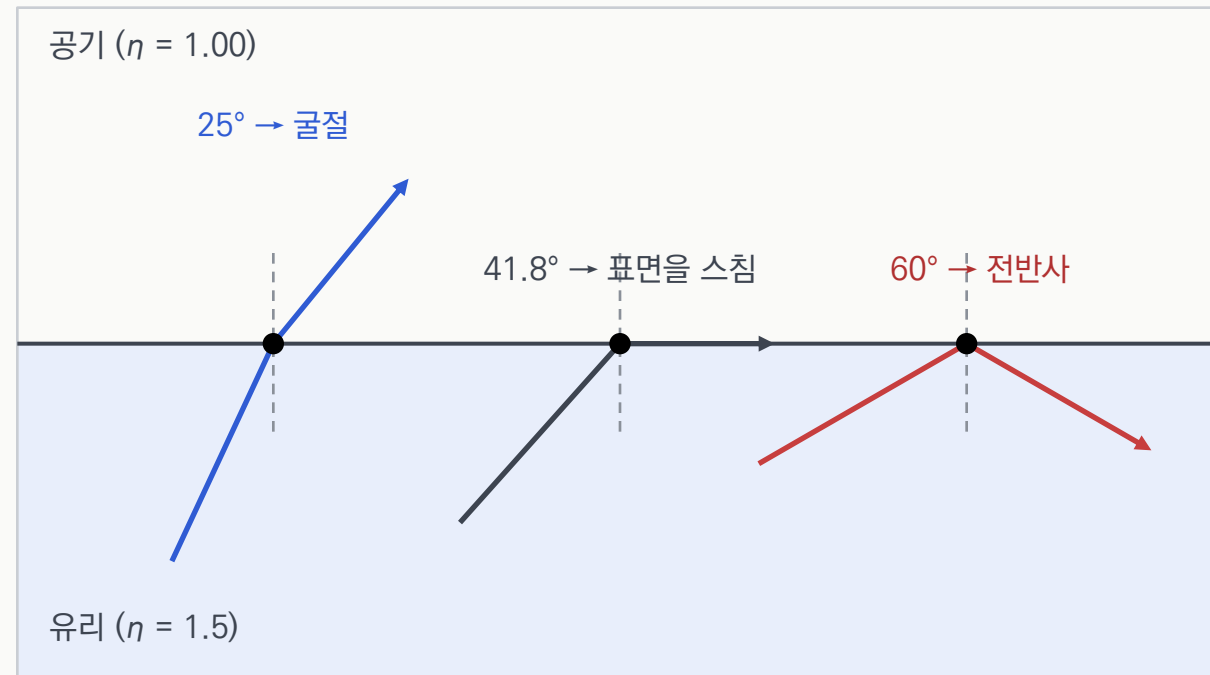
전반사: $\sin \theta'$ 가 1을 넘으면 굴절 없이 전부 반사

굴절률 높은 쪽 → 낮은 쪽, 비스듬한 입사

- $\sin \theta'$ 는 1을 넘을 수 없음
- **수식:** $\sin \theta' > 1 \rightarrow$ 굴절각 θ' 없음 (해 없음)
- **현실:** 굴절 불가 → 빛이 전부 안으로 반사 (전반사)
- **코드:** 해 없는 경우 구분 없이 진행 → 수정 필요

실제 예

- 물속에서 수면을 비스듬히 보면 **거울처럼** 보임
- 똑바로 올려다보면 **물 밖**이 보임



〈그림 46〉 유리 → 공기 경계에서 입사각에 따른 변화

굴절 가능 판정: $\text{refractionRatio} \times \sin \theta$ 가 1을 넘으면 반사

$\sin \theta$ 구하기

- 코드에 이미 있는 $\cos \theta$ 로 $\sin \theta$ 계산

$$\sin \theta = \sqrt{1 - \cos^2 \theta}$$

굴절각 존재 여부 확인

$$\sin \theta' = \text{refractionRatio} \times \sin \theta$$

- θ : 입사각, θ' : 굴절해 나갈 광선과 법선 사이 각
- 결과 1 이하 → 해당 θ' 존재 → 굴절
- 결과 1 초과 → 해당 각도 없음 → 반사

예시 ($\text{refractionRatio} = 1.5$)

입사각 30°

$$1.5 \times 0.5 = 0.75 \leq 1$$

Refract로 굴절

입사각 50°

$$1.5 \times 0.766 = 1.15 > 1$$

cannotRefract 참 → Reflect로 반사

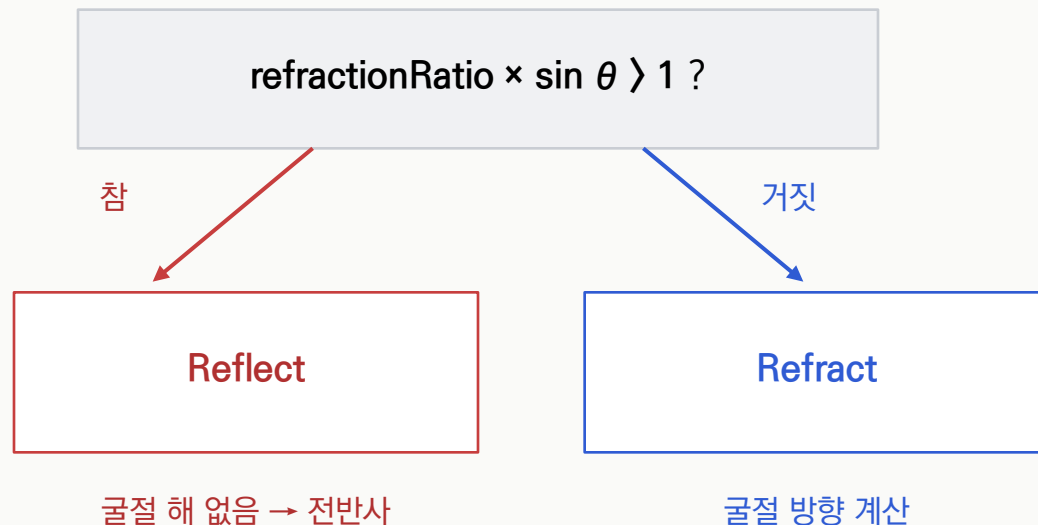
Dielectric 수정: 굴절 불가 시 Reflect, 가능 시 Refract

Dielectric::Scatter · 추가된 부분

```
// 굴절 가능 판정
const double sinTheta =
    std::sqrt(1.0 - cosTheta * cosTheta);
const bool cannotRefract =
    refractionRatio * sinTheta > 1.0;

// 분기
Vec3 direction;
if (cannotRefract)
    direction = Reflect(unitDirection, hitRecord.Normal);
else
    direction = Refract(unitDirection,
        hitRecord.Normal, refractionRatio);
```

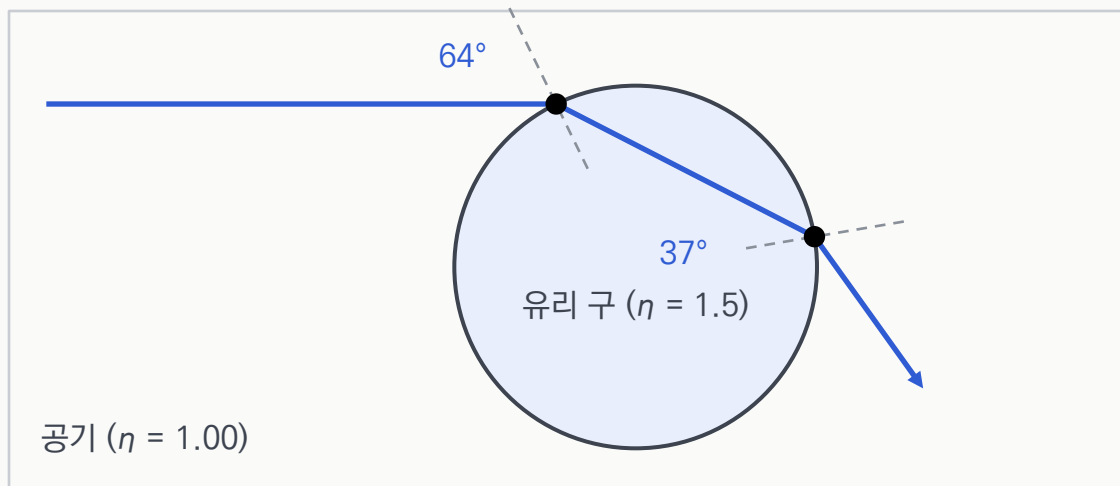
판정 흐름



공기 방울: 기존 장면에선 안 생기던 전반사 상황을 직접 구성

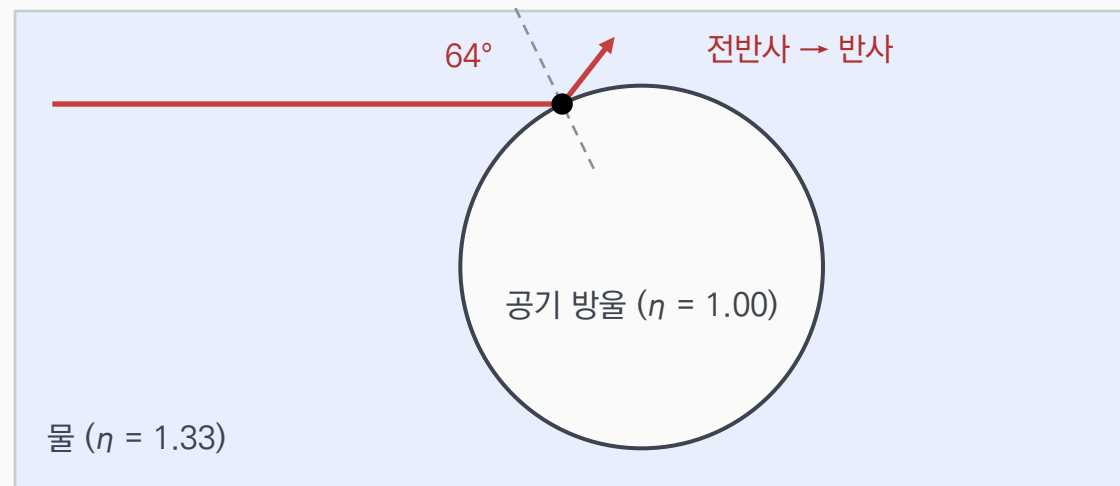
기존 world: 공기 속 유리 구 (굴절률 1.5)

- 입사 $64^\circ \rightarrow$ 안쪽 굴절 약 37°
- 나갈 때 $37^\circ <$ 임계각 $41.8^\circ \rightarrow$ 문제 상황 미발생
- 전반사 처리가 없는 코드로도 결과 차이 없음



새 world: 물속 공기 방울 (굴절률 1.00 / 1.33)

- 입사 $64^\circ >$ 임계각 약 48.8°
- $1.33 \times \sin 64^\circ \approx 1.2 > 1 \rightarrow$ 굴절 해 없음
- 수정 코드: bCannotRefract 참 $\rightarrow$ Reflect



〈그림 47〉 같은 64° 입사에서 유리 구와 공기 방울 비교

결과: 직접적인 광선은 굴절, 스치는 광선은 반사

물 속의 공기 방울 표현

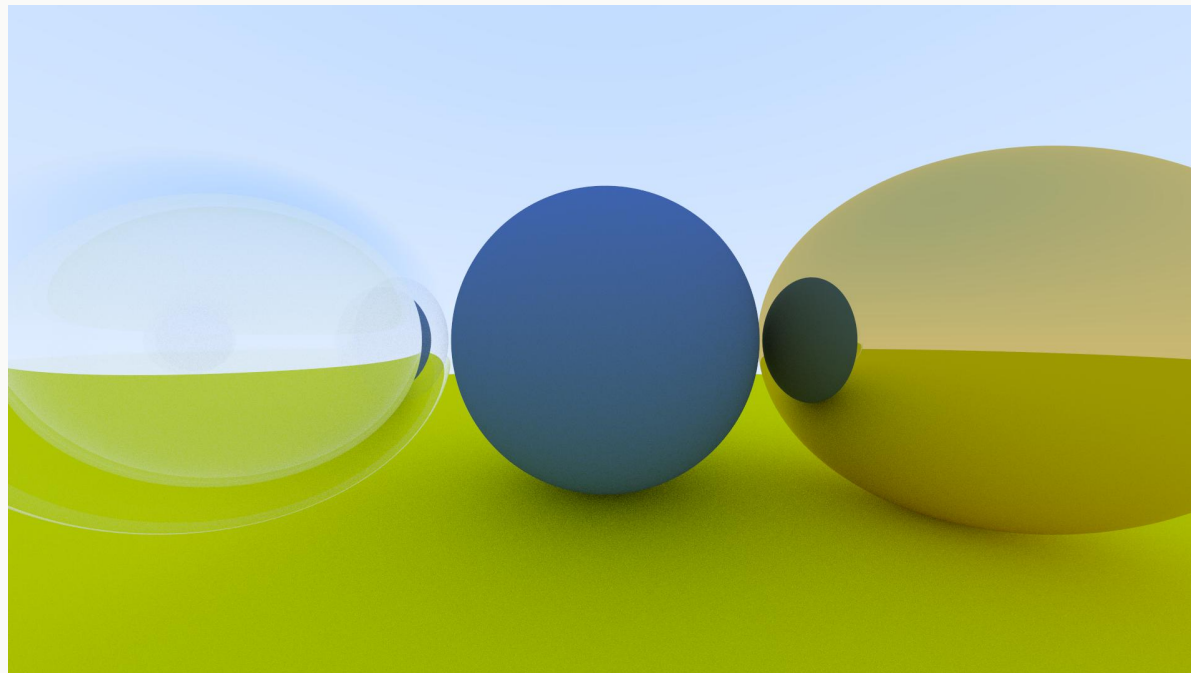
- 거의 직접적인 광선은 굴절
- 스치는 광선은 반사
- 구의 굴절률을 1.00 / 1.33으로 설정
- 물 → 공기 (방울 안으로 들어갈 때) 경계 확인 가능

```
auto materialLeft =  
    std::make_shared<Dielectric>(1.00 / 1.33);
```

상 비교 (기존 유리 구와 차이)

- 유리 구: 안쪽 굴절률 높음 → 광선 교차 → 상 뒤집힘
- 공기 방울: 안쪽 굴절률 낮음 → 광선 퍼짐 → 상 작게

결과 - 물속 공기 방울



〈그림 48〉 굴절률 1.00 / 1.33 적용 결과

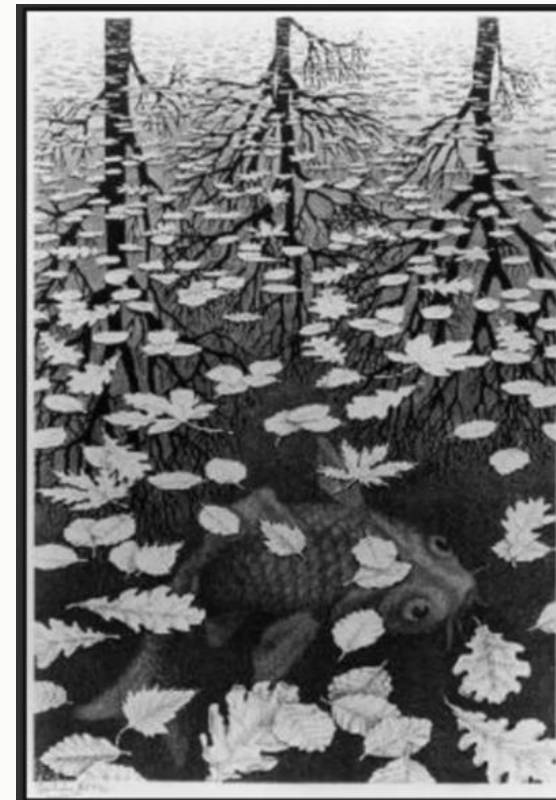
슐릭 근사: 굴절이 가능해도 각도에 따라 일부 반사

슐릭 근사 (Schlick Approximation)

- Fresnel 효과 해결 공식을 흉내 낸 근사식

적용 상황

- 비스듬할수록 반사 증가 → 수평으로 보면 대부분 반사
- 광선이 경계에 닿았고 굴절도 가능 (전반사 아님)
- 현재 코드: 이 경우 무조건 굴절
- 실제 유리: 굴절이 가능해도 각도에 따라 일부 반사



〈그림 49〉 가까운 수면은 투명, 먼 수면은 반사 (Fresnel 효과)

슐릭 근사식: 반사 확률 $R(\theta)$ 로 Reflect 선택

근사식

$$R_0 = ((\eta - \eta') / (\eta + \eta'))^2$$

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$$

반사 확률 기반 선택

- 광선 개수는 정해져 있으므로 확률 활용
- 반사 확률 기반으로 Reflect 선택

Reflectance · 슴릭 근사

```
static double Reflectance(double cos,
    double refractionIndex)
{
    // Schlick Approximation
    auto r0 = (1.0 - refractionIndex)
        / (1.0 + refractionIndex);
    r0 = r0 * r0;
    return r0 + (1.0 - r0) * std::pow((1.0 - cos), 5);
}
```

속이 빈 유리 구: 유리 껍질 안에 공기 구를 하나 더 배치

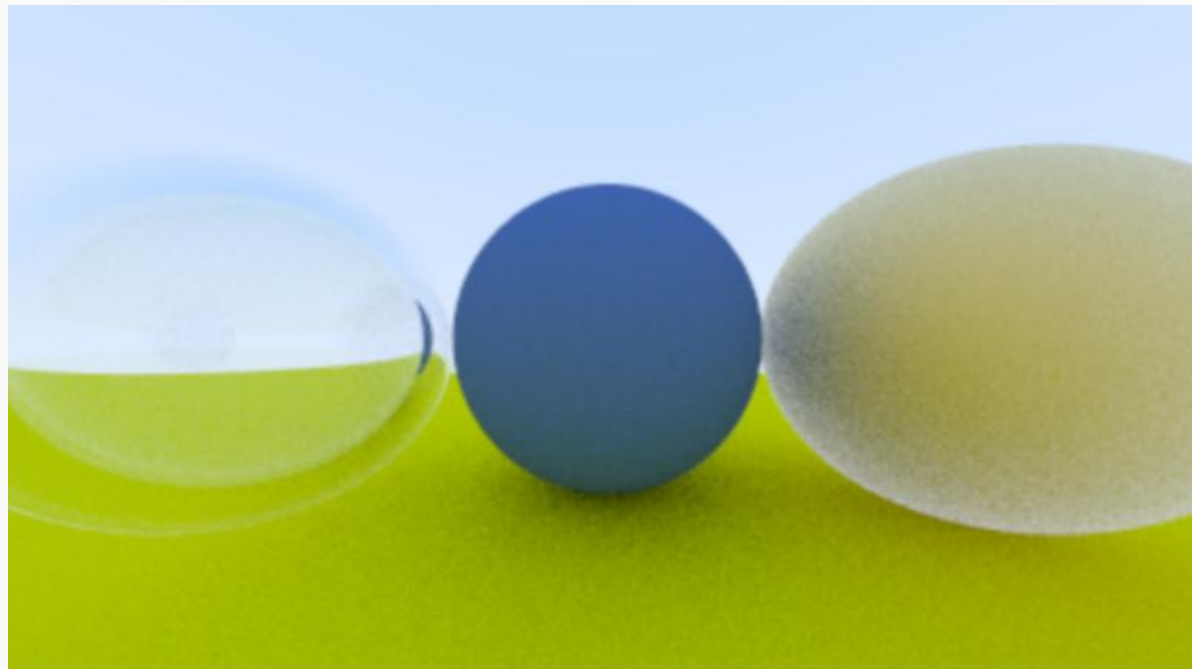
만들 것

- 두께가 있는 유리 껍질 (바깥 구)
- 유리 안에 공기로 찬 구가 하나 더 있음

경계별 refractionRatio

순서	경계	맞는 구 · 면	refractionRatio
1	공기 → 유리	바깥 구, 앞면	$1 / 1.5$
2	유리 → 안쪽 공기	안쪽 구, 앞면	$1 / (1 / 1.5) = 1.5$
3	안쪽 공기 → 유리	안쪽 구, 뒷면	$1 / 1.5$
4	유리 → 공기	바깥 구, 뒷면	1.5

결과 - 속이 빈 유리 구



〈그림 50〉 유리 껍질 안에 공기 구를 넣은 결과



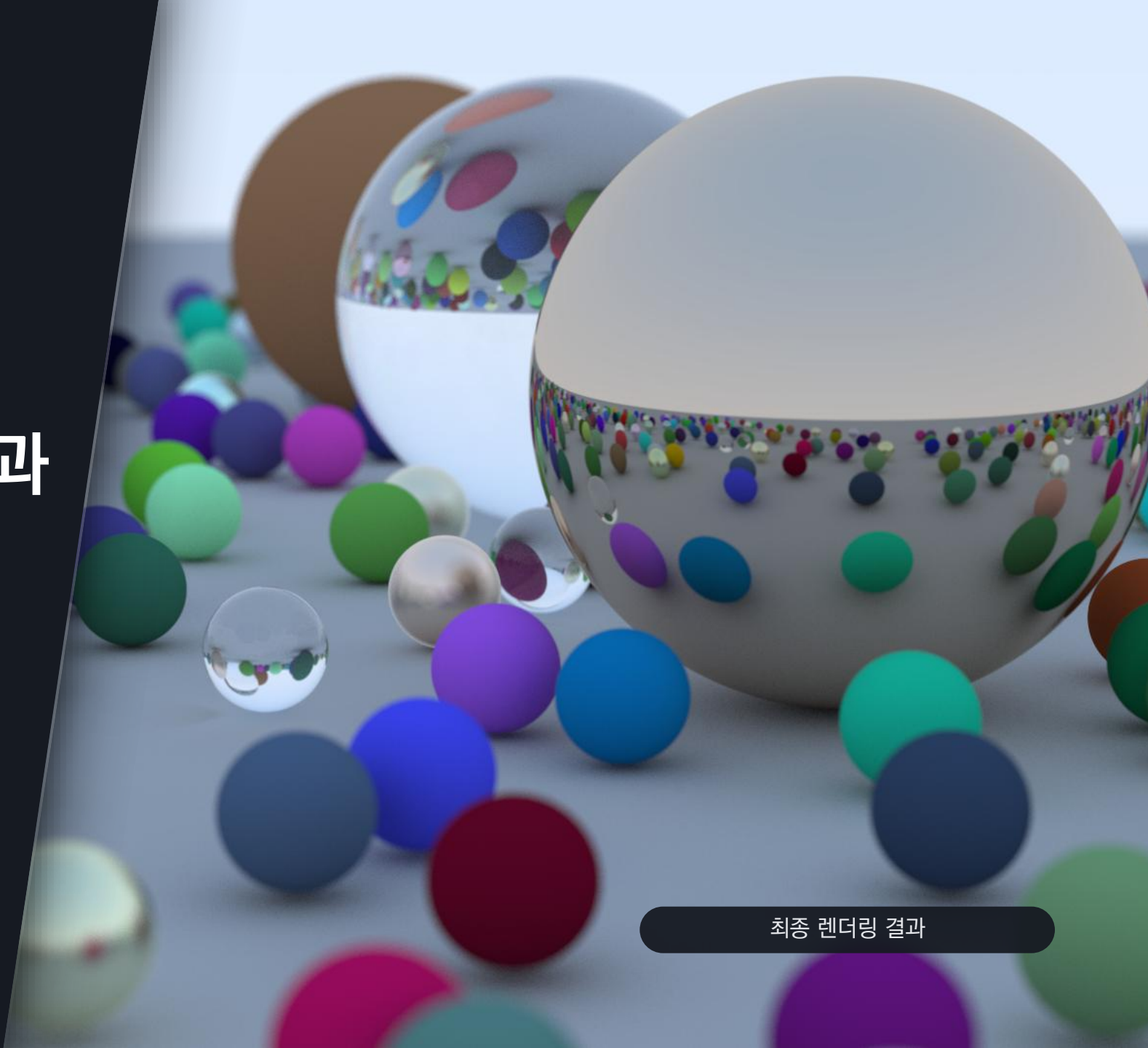
RAY TRACING IN ONE WEEKEND

PART 3

카메라 제어와 최종 결과

카메라 위치 조정부터 피사계 심도까지

Chapter 12 – 14



최종 렌더링 결과

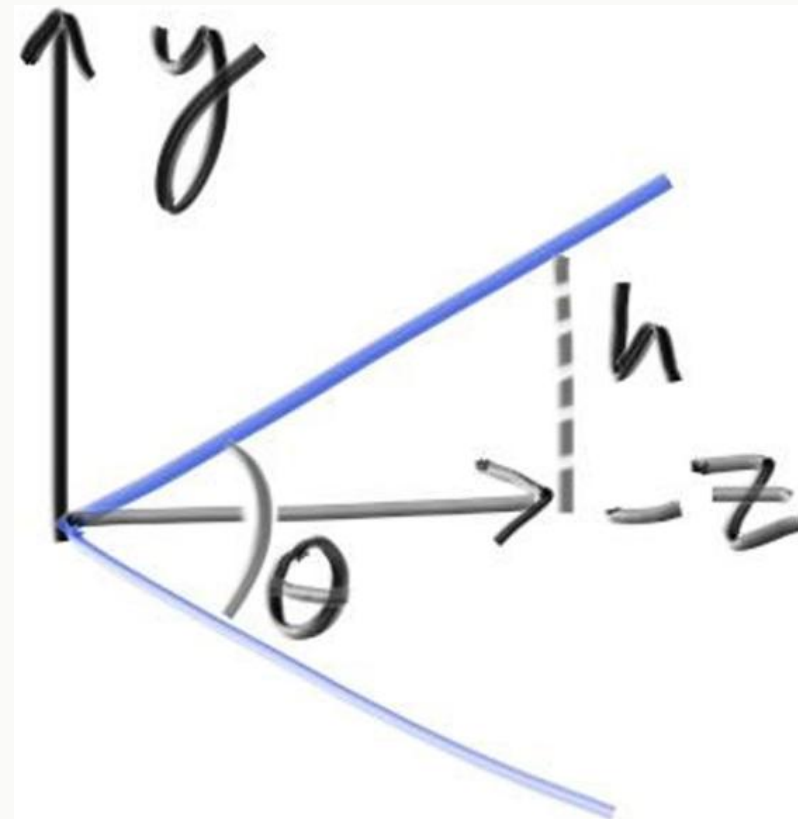
시야각(FOV): 카메라 중심과 뷰포트 위·아래 끝 사이 각

시야각 (Field of View)

- fov는 수평과 수직에서 다름
- 각도로 지정 후 사용 전 라디안으로 변환

수직 시야각 (vfov)

- 카메라 중심에서 $-z$ 로 focalLength만큼 이동
- 그 위치에 뷰포트 평면 ($z = -1$)
- 중심 $\rightarrow$ 뷰포트 위·아래 끝 선 사이 각 = $vfov(\theta)$



〈그림 51〉 카메라 중심과 뷰포트 사이 수직 시야각

뷰포트 높이: $h = \tan(\theta / 2)$ 로 시야각에 맞춰 계산

tan으로 뷰포트 높이 구하기

- 각도와 밑변을 알 때 높이를 주는 tan 사용
- focalLength = 1이면 h가 곧 높이 절반

$$h = \tan(\theta / 2)$$

$$\text{viewportHeight} = 2h \cdot \text{focalLength}$$

Camera · vfov 적용

```
// camera.h
double vfov = 90; // 수직 시야각

void Initialize()
{
    double focalLength = 1.0;
    double theta = DegreesToRadians(vfov);
    double h = std::tan(theta / 2);
    double viewportHeight = 2.0 * h * focalLength;
    double viewportWidth = viewportHeight *
        (double(mImageWidth) / mImageHeight);
    ...
}
```

장면 변경: 파란 구와 빨간 구를 $vfov = 90$ 으로 렌더링

main · 장면 변경

- 렌더링 장면 자체를 새로 구성

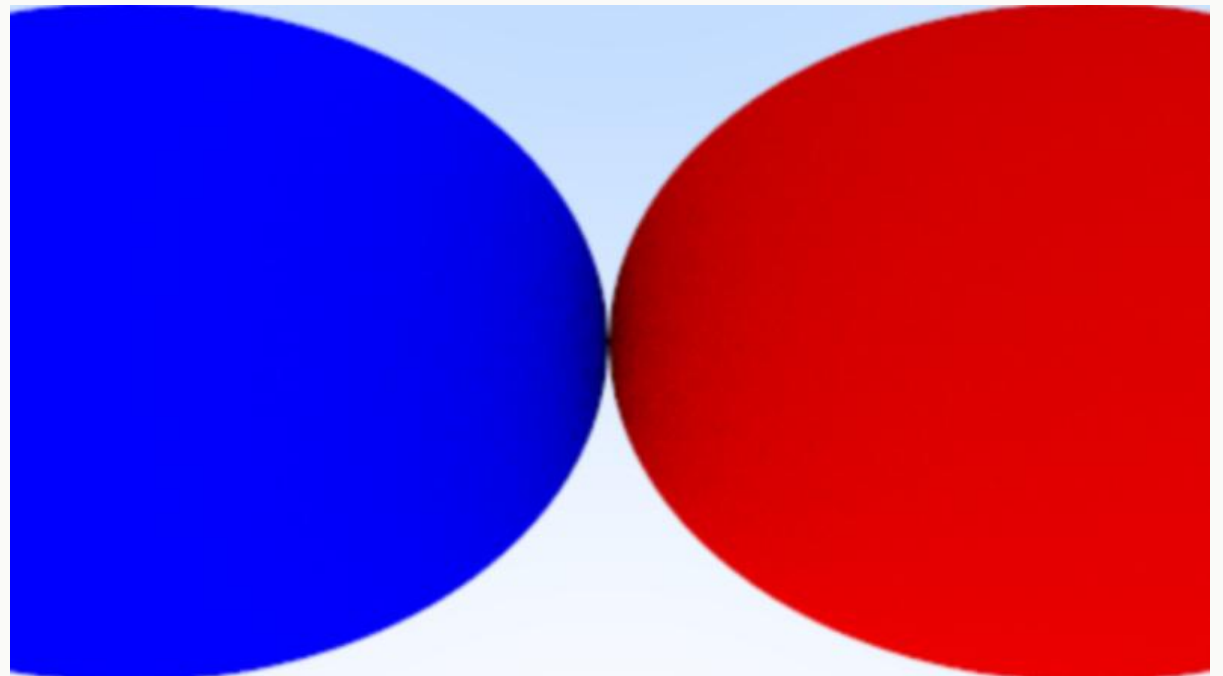
```
auto R = std::cos(Pi / 4);

auto materialLeft =
    make_shared<Lambertian>(Color(0, 0, 1));
auto materialRight =
    make_shared<Lambertian>(Color(1, 0, 0));

world.Add(make_shared<Sphere>(
    Point3(-R, 0, -1), R, materialLeft));
world.Add(make_shared<Sphere>(
    Point3(R, 0, -1), R, materialRight));
```

```
camera.vfov = 90;
```

결과 - $vfov = 90$



〈그림 52〉 $vfov = 90$ 렌더링 결과

카메라 배치: lookfrom · lookat · vup으로 위치와 방향 지정

위치와 바라보는 점

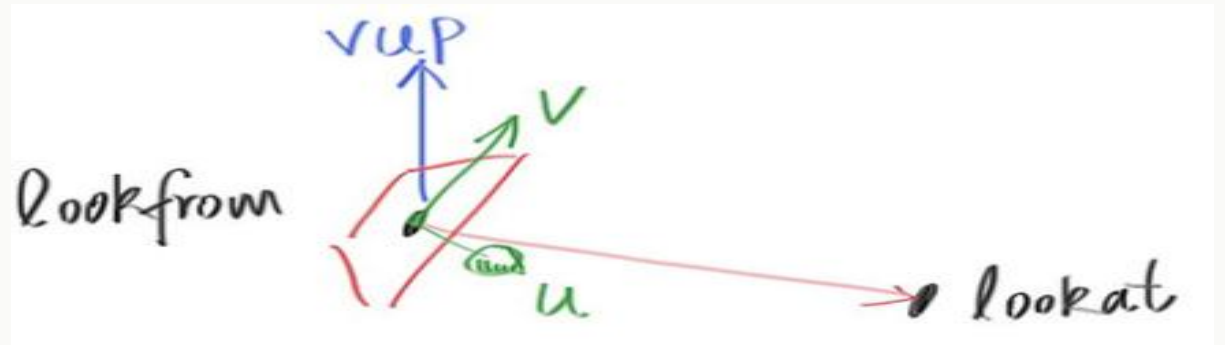
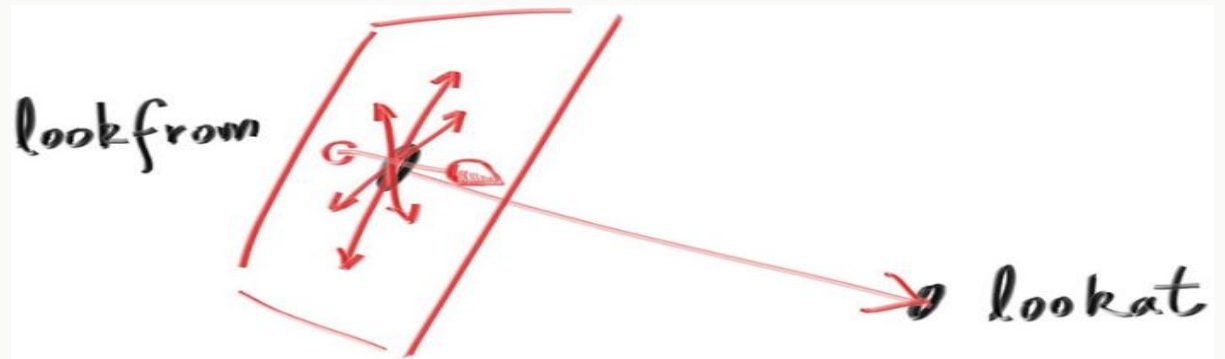
- lookfrom: 카메라를 배치하는 위치
- lookat: 카메라가 바라보는 점

view direction = lookfrom $\rightarrow$ lookat

카메라 위쪽 벡터 (vup)

- 카메라가 회전하지 않은 상태를 가정해 직접 지정
- view direction과 **평행만 아니면** 지정 가능
- 회전으로 바라보는 방향과 같아지면 vup 수정 필요

카메라 배치



〈그림 53〉 lookfrom · lookat · vup 설정

카메라 기저 $u \cdot v \cdot w$: 외적으로 서로 직각인 세 축 계산

카메라 좌표축

축	가리키는 곳	기존 월드축
u	카메라의 오른쪽	$+x$
v	카메라의 위쪽	$+y$
w	바라보는 방향의 반대 (카메라 뒤쪽)	$+z$



〈그림 54〉 카메라 기저 벡터 $u \cdot v \cdot w$

축을 구하는 순서

① w : 바라보는 점 $\rightarrow$ 카메라 방향

$$w = (\text{lookfrom} - \text{lookat}) / |\text{lookfrom} - \text{lookat}|$$

② u : vup 과 w 둘 다에 직각 (외적 길이가 1이 아닐 수 있어 정규화)

$$u = (vup \times w) / |vup \times w|$$

③ v : w, u 가 길이 1이고 직각 $\rightarrow$ 정규화 불필요

$$v = w \times u$$

결과: 같은 위치에서 $vfov = 90$ 은 넓게, $vfov = 20$ 은 좁게 보임

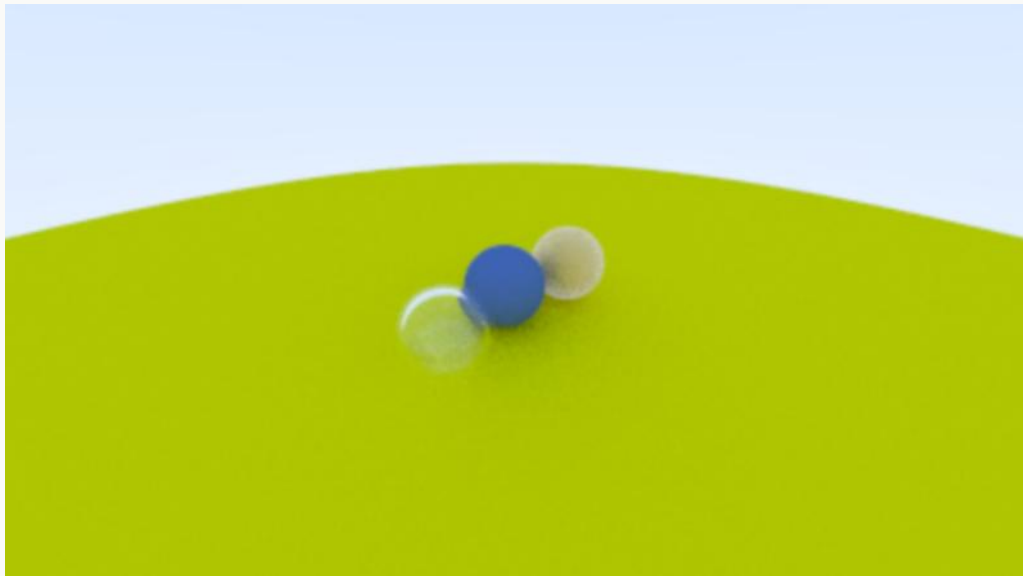
Initialize 변경점 · 12-5의 $u \cdot v \cdot w$ 적용

카메라 중심
 $mCenter = lookfrom$

초점 길이
 $focalLength = |lookfrom - lookat|$

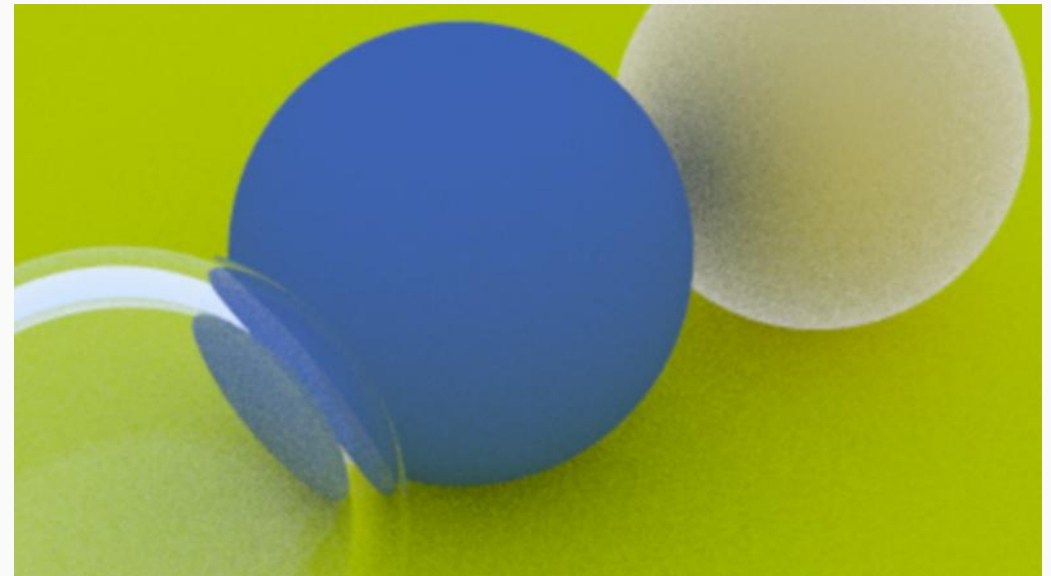
뷰포트 이동 방향
 $-Z \rightarrow -focalLength \cdot w$

넓은 시야 · $vfov = 90$



〈그림 55〉 넓은 시야 렌더링 결과 ($vfov = 90$)

좁은 시야 · $vfov = 20$



〈그림 56〉 좁은 시야 렌더링 결과 ($vfov = 20$)

초점 흐림: 초점 거리에서 벗어난 물체가 흐려 보이는 현상

초점 흐림 (Defocus Blur)

- 실제 카메라는 빛을 모으려고 **큰 구멍** 필요
- 큰 구멍 때문에 **초점 흐림** 발생
- 구멍 앞 **렌즈** → 초점이 맞는 특정 거리 생성
- 초점 거리와 안 맞으면 흐려 보임

초점 거리 vs 초점 길이

구분	기준
초점 거리 (focus distance)	카메라 중심 ↔ 초점이 맞는 평면
초점 길이 (focal length)	카메라 중심 ↔ 이미지 평면

- 본 프로젝트는 이미지 평면을 초점 거리에 배치
=> 두 거리 동일

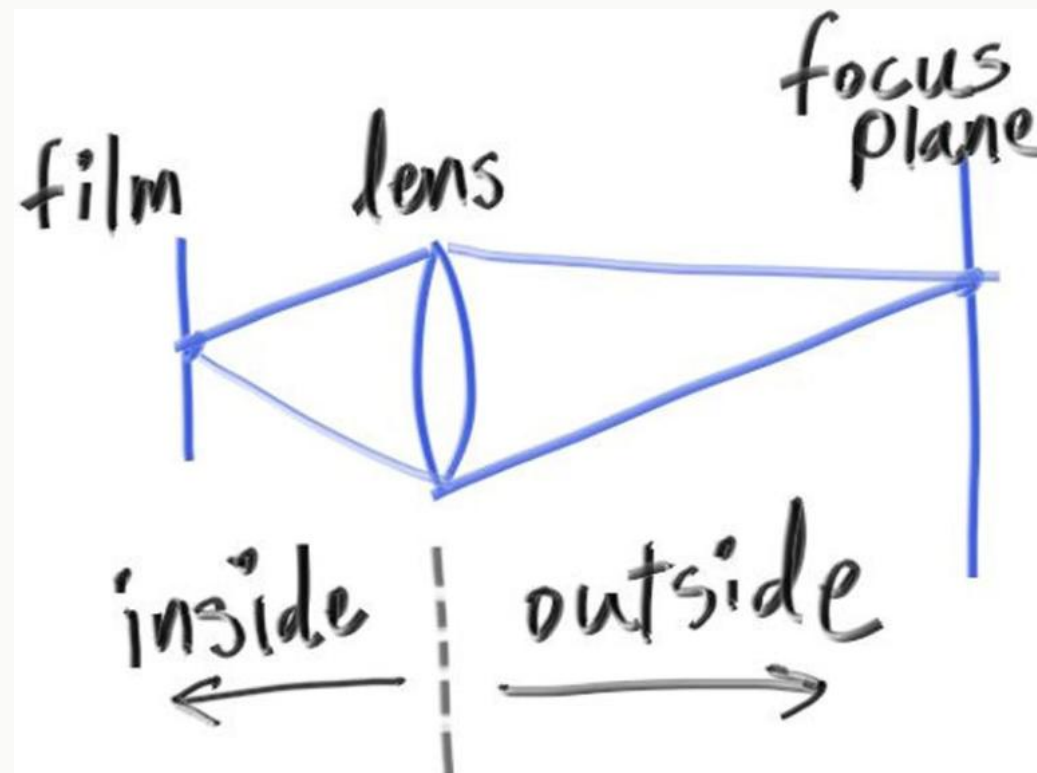
가상 카메라: 얇은 렌즈 근사로 센서 · 렌즈 · 조리개 시뮬레이션

실제 카메라와 가상 카메라

- 실제 카메라: 빛의 양 조절을 위해 조리개 조절
- 조리개가 클수록 초점 밖 물체가 더 흐림
- 가상 카메라: 완벽한 센서, 추가 빛 불필요
- 초점 흐림을 원할 때만 조리개 사용

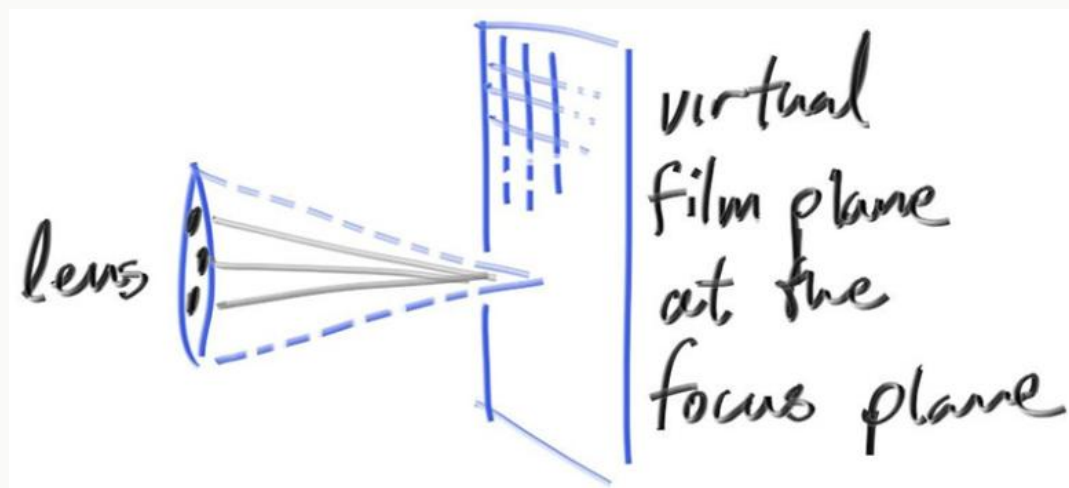
얇은 렌즈 근사 (Thin Lens Approximation)

- 센서, 렌즈, 조리개를 그대로 사용하면 상이 뒤집혀 복잡함
 - 일반적으로 얇은 렌즈 근사 사용
- => 센서 없이 렌즈 원판 위 점에서 초점 평면으로 광선 발사



〈그림 58〉 얇은 렌즈 근사 모델

가상 카메라 구현: 렌즈 위 무작위 점에서 광선 발사



〈그림 59〉 가상 카메라의 렌즈와 초점 평면

가상 카메라 구현

1. 초점 평면은 카메라 시야 방향에 직교
2. 초점 거리 = 초점 길이
3. 뷰포트는 초점 평면 위, 시야 방향 벡터가 중심
4. 픽셀 위치 그리드는 뷰포트 내부
5. 샘플 위치는 현재 픽셀 주변에서 무작위 선택
6. 렌즈 위 무작위 지점 → 샘플 위치로 광선 발사

흐려지는 이유

- 광선 출발점이 한 점이 아닌 **렌즈 위 무작위 점**
- 광선은 모두 초점 평면 위 같은 점을 향함 → 초점 평면 위 물체는 선명
- 초점 평면 앞뒤의 물체는 광선마다 닿는 점이 달라 흐림

렌즈 원판 반지름: focus_dist와 defocus_angle로 계산

렌즈 원판 반지름 구하기

- **밀변:** focus_dist (뷰포트 중심 ↔ 카메라 중심)
- **높이:** 렌즈 원판 반지름
- **꼭짓점 각:** defocus_angle / 2

$$\text{defocusRadius} = \text{focus_dist} \cdot \tan(\text{defocus_angle} / 2)$$

Camera::Initialize · 렌즈 원판

```
double defocusRadius = focus_dist *  
    std::tan(DegreesToRadians(defocus_angle * 0.5));  
  
// 렌즈 원판 오른쪽 반지름 벡터  
mDefocusDiskU = u * defocusRadius;  
// 렌즈 원판 위쪽 반지름 벡터  
mDefocusDiskV = v * defocusRadius;
```

렌즈 원판 샘플링: 단위 원의 무작위 점을 원판 크기로 조정

단위 원 → 렌즈 원판

- 렌즈 원판 위 무작위 점 → 단위 원 활용
- 지정한 원판 크기로 조정

```
// RandomInUnitDisk
inline Vec3 RandomInUnitDisk()
{
    while (true)
    {
        Vec3 p = Vec3(RandomDouble(-1.0, 1.0),
                      RandomDouble(-1.0, 1.0), 0.0);
        if (p.LengthSquared() < 1.0)
        {
            return p;
        }
    }
}
```

Camera::DefocusDiskSample

```
Point3 DefocusDiskSample() const
{
    Vec3 point = RandomInUnitDisk();
    return mCenter + (point.X() * mDefocusDiskU)
               + (point.Y() * mDefocusDiskV);
}
```

Camera::GetRay

```
Point3 rayOrigin = (defocus_angle <= 0.0)
    ? mCenter : DefocusDiskSample();
```

뷰포트 이동: 초점 길이 위치에서 초점 평면(focus_dist)으로

추가된 멤버 변수

```
// 렌즈 원판 지름이 뷰포트 중심에서 보이는 각도
// 0이면 흐림 없음
double defocus_angle = 0;

// 카메라 중심(lookfrom)에서 초점 평면까지 거리
double focus_dist = 10;
```

뷰포트 위치 변경

- 기존: 초점 길이 위치에 뷰포트 배치
- 변경: 초점 평면으로 뷰포트 이동

Camera::Initialize 일부 · 뷰포트

```
double theta = DegreesToRadians(vfov);
double h = std::tan(theta / 2);
double viewportHeight = 2.0 * h * focus_dist;
double viewportWidth = viewportHeight *
    (static_cast<double>(imageWidth) / mImageHeight);

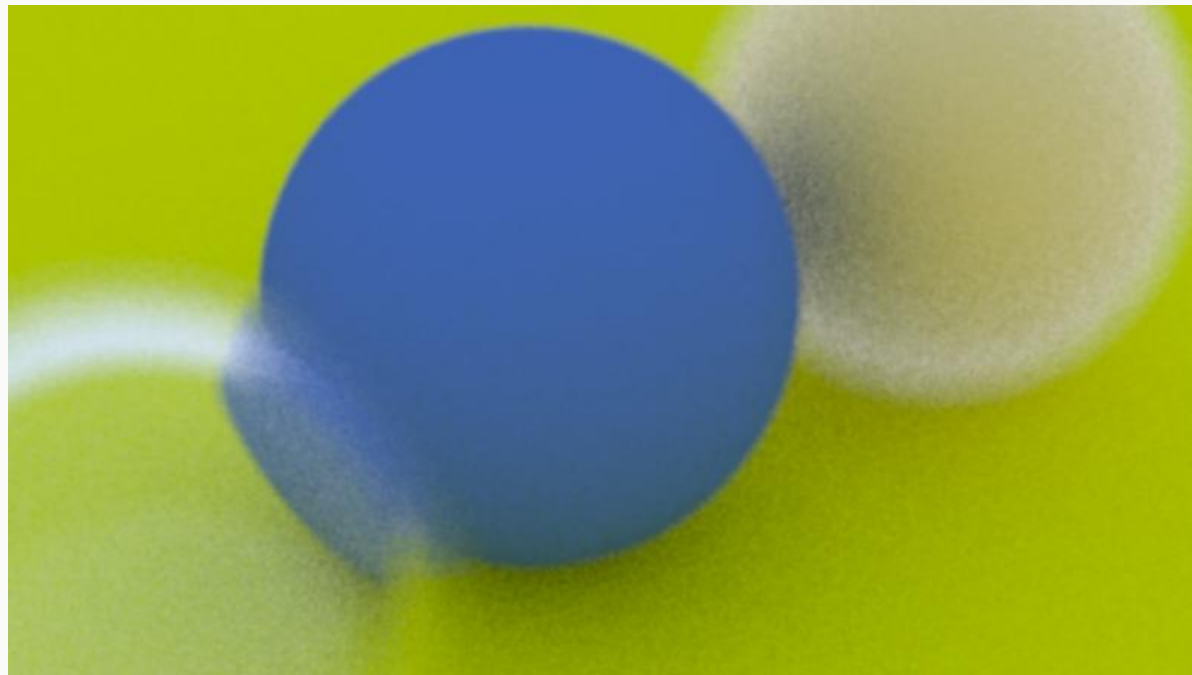
Point3 viewportUpperLeft = mCenter
    - (focus_dist * w)
    - viewportU / 2.0 - viewportV / 2.0;
```

초점 흐림 렌더링 결과

카메라 설정

```
// main  
camera.defocus_angle = 10.0;  
camera.focus_dist = 3.4;
```

결과



〈그림 60〉 초점 흐림 렌더링 결과

최종 장면: 무작위 구 배치와 카메라 설정으로 렌더링

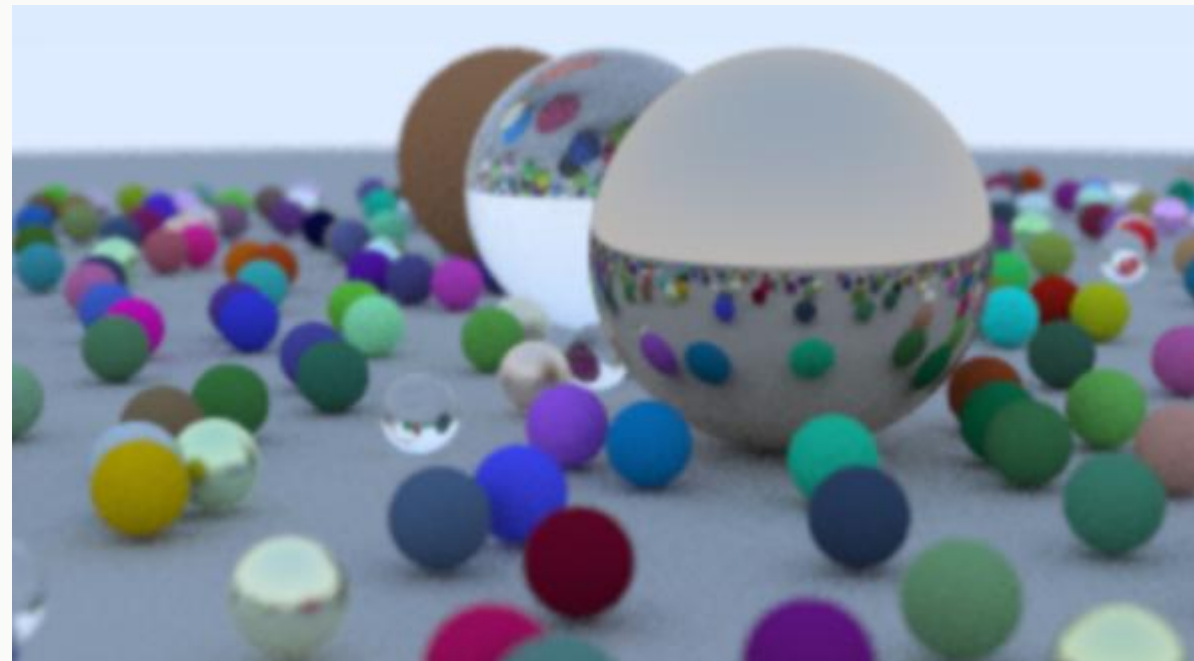
장면 구성

- 땅: 반지름 1000 회색 Lambertian 구
- 작은 구: 22×22 격자에 무작위 배치
- 재질: 확산 80% · 금속 15% · 유리 5%
- 큰 구 3개: Dielectric · Lambertian · Metal

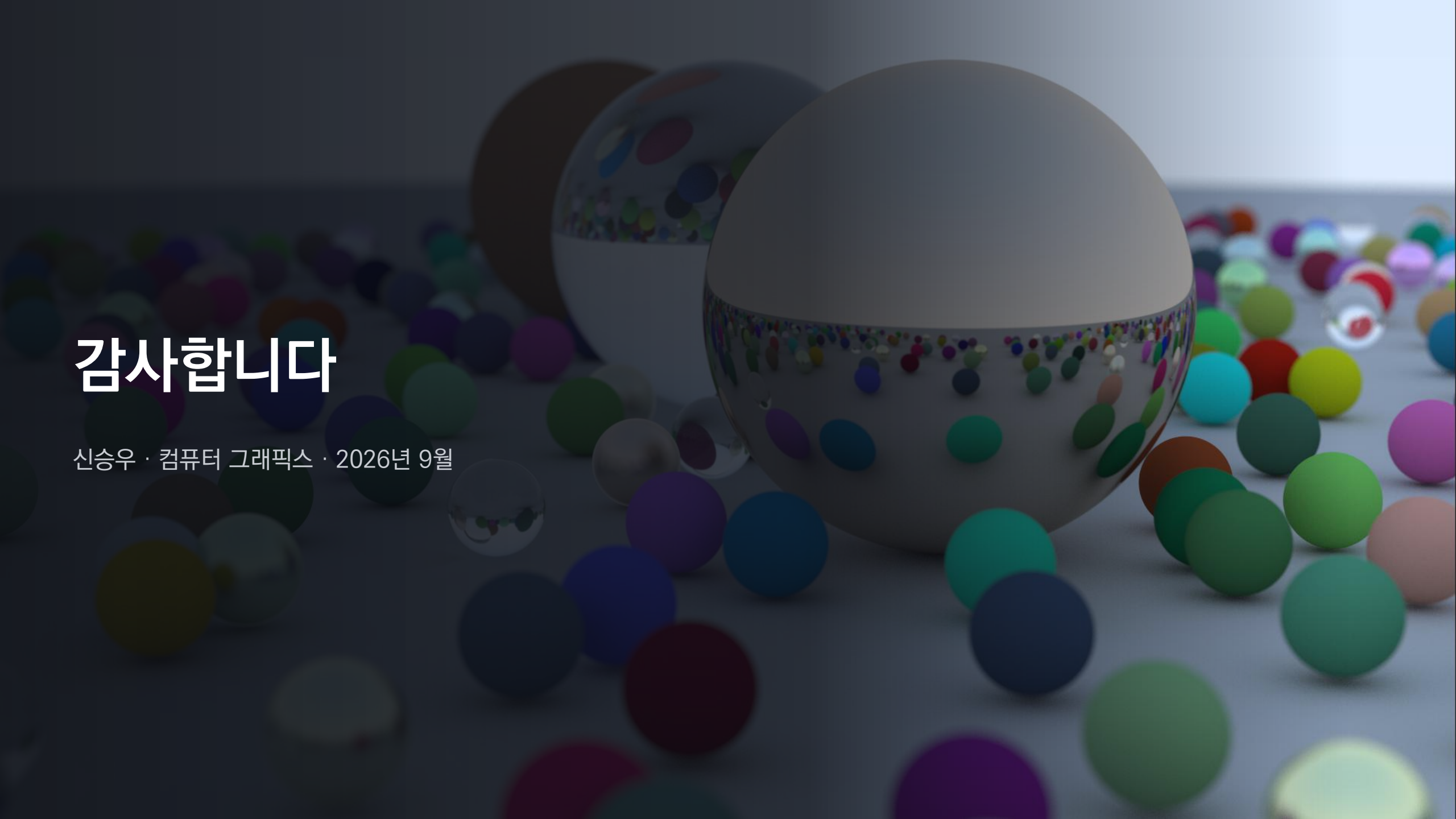
카메라 설정

- 시점: lookfrom (13, 2, 3) → lookat (0, 0, 0)
- vfov 20 · defocus_angle 0.6 · focus_dist 10

결과



〈그림 61〉 최종 장면 렌더링 결과



감사합니다

신승우 · 컴퓨터 그래픽스 · 2026년 9월

PPM P3 · P6 차이: 색상 값의 표현 방식

	P3	P6
방식	ASCII	바이너리
255 표현	문자 2, 5, 5 → 50, 53, 53	11111111

C++ 기능과 공통 상수: shared_ptr · vector · constexpr

새로운 C++ 기능

기능	설명
shared_ptr	참조 카운트 기반 스마트 포인터
make_shared<T>()	shared_ptr 생성
vector	크기가 자동으로 늘어나는 배열
push_back()	vector에 원소 추가

- shared_ptr: 마지막 참조가 사라지면 자동 해제

```
std::shared_ptr<Sphere> spherePointer =
    std::make_shared<Sphere>(Point(0.0, 0.0, 0.0), 1.0);
std::vector<std::shared_ptr<Hittable>> mObjects;
mObjects.push_back(object);
```

공통 상수 및 유틸리티 함수

- Infinity, Pi, DegreesToRadians를 RTWeekend.h에 모음

```
constexpr double Infinity =
    std::numeric_limits<double>::infinity();
constexpr double Pi = 3.1415926535897932385;
```

추상화 누수: 원점 근처 점의 길이² 언더플로우 방지

부동소수점 추상화 누수

- 유한한 정밀도로 수학적 식이 깨지는 현상
- 아주 작은 값의 제곱 → 언더플로우로 0
- 세 좌표가 모두 작으면 벡터 길이 0

문제

- p 가 원점에 극도로 가까우면 길이² = 0
- 길이로 나눌 때 0으로 나누기 발생

RandomUnitVector 수정

```
if (1e-160 < lengthSquared && lengthSquared <= 1.0)
{
    return p / std::sqrt(lengthSquared);
}
```

확률 기반 반사: 난수가 반사 확률보다 작으면 Reflect

난수 구간 (반사 확률 0.04 예시)

난수가 여기 떨어지면 **Reflect** (4%)



나머지 구간 → **Refract** (96%)

Dielectric::Scatter · 반사 판정

```
if (cannotRefract ||  
    Reflectance(cosTheta, refractionRatio) > RandomDouble())  
{  
    direction = Reflect(unitDirection, hitRecord.Normal);  
}  
else  
{  
    direction = Refract(unitDirection, hitRecord.Normal,  
        refractionRatio, cosTheta);  
}
```

R₀ 변환: η' 로 나눈 식을 코드에 그대로 사용

코드의 R₀ 표기

- 코드는 η' 로 나눈 상태의 식 활용 ($r = \eta / \eta'$)
- 제공이므로 부호는 의미 없음
- 분자의 좌우 위치가 바뀌어 표기

코드의 최종 식

$$R_0 = ((1 - \eta) / (1 + \eta))^2$$

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$$

η' 로 나누기

$$\begin{aligned} & (\eta - \eta') / (\eta + \eta') \\ &= (\eta/\eta' - 1) / (\eta/\eta' + 1) = (r - 1) / (r + 1) \end{aligned}$$

부호 정리

$$\begin{aligned} & ((1 - r) / (1 + r))^2 = (- (r - 1) / (r + 1))^2 \\ &= (-1)^2 ((r - 1) / (r + 1))^2 = ((r - 1) / (r + 1))^2 \end{aligned}$$

속이 빈 유리 구: 광선 경로와 재질 · 구 배치 코드

광선 경로

- 바깥 구에 충돌 → 굴절
- 안쪽 구에 충돌 → 굴절 후 내부 공기 통과
- 안쪽 구 내부 표면 → 다시 굴절
- 바깥 구 내부 표면 → 굴절 후 대기로 빠져나감

main · 재질과 구 배치

```
auto materialGround = std::make_shared<Lambertian>(
    Color(0.8, 0.8, 0.0));
auto materialCenter = std::make_shared<Lambertian>(
    Color(0.1, 0.2, 0.5));
auto materialLeft = std::make_shared<Dielectric>(1.50);
auto materialBubble = std::make_shared<Dielectric>(
    1.0 / 1.50);
auto materialRight = std::make_shared<Metal>(
    Color(0.8, 0.6, 0.2), 1.0);

world.Add(std::make_shared<Sphere>(Point3(0.0, -100.5, -1.0),
    100.0, materialGround));
world.Add(std::make_shared<Sphere>(Point3(0.0, 0.0, -1.2),
    0.5, materialCenter));
world.Add(std::make_shared<Sphere>(Point3(-1.0, 0.0, -1.0),
    0.5, materialLeft));
world.Add(std::make_shared<Sphere>(Point3(-1.0, 0.0, -1.0),
    0.4, materialBubble));
world.Add(std::make_shared<Sphere>(Point3(1.0, 0.0, -1.0),
    0.5, materialRight));
```

Initialize 수정: lookfrom 기준으로 $u \cdot v \cdot w$ 축 계산

추가된 멤버 변수

```
double vfov = 90; // 수직 시야각
Point3 lookfrom = Point3(0, 0, 0);
Point3 lookat = Point3(0, 0, -1);
Vec3 vup = Vec3(0, 1, 0); // 카메라의 위쪽 방향
Vec3 u, v, w; // 카메라 프레임 기저 벡터
```

뷰포트 좌상단 계산 변경

```
// 기존: 월드 -Z 방향으로 focalLength만큼 이동
mCenter = Vec3(0, 0, focalLength)
```

```
// 수정: 카메라가 바라보는 방향으로 focalLength만큼 이동
mCenter = focalLength * w
```

Camera::Initialize

```
mCenter = lookfrom;
double focalLength = (lookfrom - lookat).Length();

double theta = DegreesToRadians(vfov);
double h = std::tan(theta / 2);
double viewportHeight = 2.0 * h * focalLength;
double viewportWidth = viewportHeight *
    (double(mImageWidth) / mImageHeight);

w = UnitVector(lookfrom - lookat);
u = UnitVector(Cross(vup, w));
v = Cross(w, u);

Vec3 viewportU = viewportWidth * u;
Vec3 viewportV = viewportHeight * -v;

Point3 viewportUpperLeft = mCenter
    - (focalLength * w)
    - viewportU / 2.0 - viewportV / 2.0;
```

카메라 변수 확인: 잘못 수정하면 렌더링 오류 발생

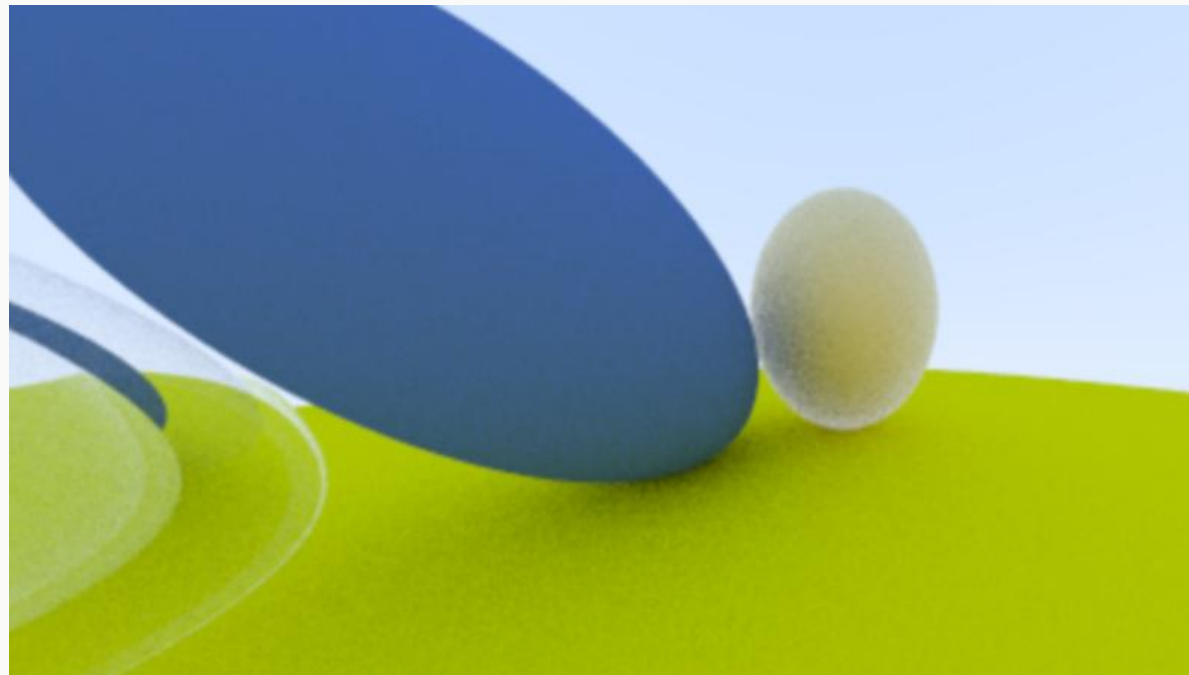
참고

- 잘못 수정하면 렌더링 오류 발생
- 카메라 변수 꼭 제대로 확인

뷰포트 좌상단 · 수정한 부분

```
// 수정: 카메라가 바라보는 방향으로 focalLength만큼 이동  
mCenter - focalLength * w
```

잘못 수정한 경우



〈그림 57〉 카메라 변수를 잘못 수정한 렌더링 결과

최종 장면 코드 1- 땅과 작은 구 무작위 배치

땅 배치

```
HittableList world;
std::shared_ptr<Lambertian> groundMaterial =
    std::make_shared<Lambertian>(Color(0.5, 0.5, 0.5));
world.Add(std::make_shared<Sphere>(
    Point3(0.0, -1000.0, 0.0), 1000.0, groundMaterial));
```

작은 구 무작위 배치

```
for (int gridX = -11; gridX < 11; ++gridX)
{
    for (int gridZ = -11; gridZ < 11; ++gridZ)
    {
        double materialSelector = RandomDouble();
        Point3 center(gridX + 0.9 * RandomDouble(), 0.2,
                      gridZ + 0.9 * RandomDouble());
        if ((center - Point3(4.0, 0.2, 0.0))
            .Length() > 0.9)
        {
            std::shared_ptr<Material> sphereMaterial;
```

작은 구 무작위 배치 (이어서)

```
if (materialSelector < 0.8)
{
    Color albedo = Color::Random() * Color::Random();
    sphereMaterial = std::make_shared<Lambertian>(albedo);
}
else if (materialSelector < 0.95)
{
    Color albedo = Color::Random(0.5, 1.0);
    double fuzz = RandomDouble(0.0, 0.5);
    sphereMaterial = std::make_shared<Metal>(albedo, fuzz);
}
else
{
    sphereMaterial = std::make_shared<Dielectric>(1.5);
}
world.Add(std::make_shared<Sphere>(
    center, 0.2, sphereMaterial));
}
}
```

최종 장면 코드 2- 큰 구 세 개와 카메라 배치

큰 구 세 개 배치

```
std::shared_ptr<Dielectric> material1 =
    std::make_shared<Dielectric>(1.5);
world.Add(std::make_shared<Sphere>(
    Point3(0.0, 1.0, 0.0), 1.0, material1));

std::shared_ptr<Lambertian> material2 =
    std::make_shared<Lambertian>(Color(0.4, 0.2, 0.1));
world.Add(std::make_shared<Sphere>(
    Point3(-4.0, 1.0, 0.0), 1.0, material2));

std::shared_ptr<Metal> material3 =
    std::make_shared<Metal>(Color(0.7, 0.6, 0.5), 0.0);
world.Add(std::make_shared<Sphere>(
    Point3(4.0, 1.0, 0.0), 1.0, material3));
```

카메라 배치

```
Camera camera;

camera.mAspectRatio = 16.0 / 9.0;
camera.mImageWidth = 300;
camera.mSamplesPerPixel = 100;
camera.mMaxDepth = 50;

camera.vfov = 20.0;
camera.lookfrom = Point3(13.0, 2.0, 3.0);
camera.lookat = Point3(0.0, 0.0, 0.0);
camera.vup = Vec3(0.0, 1.0, 0.0);

camera.defocus_angle = 0.6;
camera.focus_dist = 10.0;

camera.Render(world);
```